These data structures are initialized to false. To prove that the mutual exclusion requirement is met, we note that process P_i can enter its critical section only if either waiting[i]= = false or key = = false. The value of key can become false only if TestAndSet is executed. The first process to execute the TestAndSet instruction will find key= =false; all others must wait. The variable waiting[i] can only become false if another process leaves its critical section; only one waiting[i] is set to false, maintaining the mutual exclusion requirement.

To prove the progress requirement is met, we note that the arguments presented for mutual exclusion also apply here, since a process exiting the critical section either sets lock to false or sets waiting[j] to false. Both allow a process that is waiting to enter its critical section to proceed.

To prove that the bounded waiting requirement is met, we note that, when a process leaves its critical section, it scans the array waiting in the cyclic ordering (i+1, i+2, ..., n-1, 0, 1, ..., i-1). It designates the first process it sees that is in its entry section with waiting[j]=true as the next one to enter its critical section. Any process waiting to do so will enter its critical section within n-1 turns.

Semaphores

Hardware solutions to synchronization problems are not easy to generalize to more complex problems. To overcome this difficulty we can use a synchronization tool called a semaphore. A **semaphore** S is an integer variable that, apart from initialization is accessible only through two standard atomic operations: wait and signal. These operations were originally termed P (for wait) and V (for signal). The classical definitions of wait and signal are:

```
wait(S) {
   while(S<=0)
   ;// no op
   S--;
}
```

```
signal(S) {
   S++;
}
```

Modifications to the integer value of the semaphore in the wait and signal operations must be executed indivisibly. That is, when one process is updating the value of a semaphore, other processes cannot simultaneously modify that same semaphore value. In addition, in the case of the wait(S), the testing of the integer value of S (S<=0) and its possible modification (S--) must also be executed without interruption.

We can use semaphores to deal with the n-process critical section problem. The n processes share a semaphore, **mutex** (standing for mutual exclusion) initialized to 1. Each process P_i is organized as follows:

CamScanner

```
do
{
        wait(mutex);
            Critical section
        signal(mutex);
            Remainder section
} while(1);
```

As was the case with the hardware-based solutions, this is not a good solution because even though it satisfies mutual exclusion and progress, it does not satisfy bounded wait.

In a uni-processor environment, to ensure atomic execution, while executing wait and signal, interrupts can be disabled. In case of a multi-processor environment, to ensure atomic execution is one can lock the data bus, or use a soft solution such as the Bakery algorithm.

MOST Imp-

The main disadvantage of the semaphore discussed in the previous section is that it requires **busy waiting**. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. This continual looping is clearly a problem in a real multiprogramming system, where a single CPU is shared among many processes. Busy waiting wastes CPU cycles that some other process may be able to use productively. This type of semaphore is also called a **spinlock** (because the process spins while waiting for the lock). Spinlocks are useful in multiprocessor systems. The advantage of a spinlock is that no context switch is required when a process must wait on a lock, and a context switch may take considerable time. This is, spinlocks are useful when they are expected to be held for short times. The definition of semaphore should be modified to eliminate busy waiting. We will discuss the modified definition of semaphore in the next lecture.

CamScanner

Operating Systems
Lecture No. 23

Reading Material

- Chapter 7 of the textbook
- Lecture 23 on Virtual TV

Summary

- Busy waiting
- New definition of semaphore
- Process synchronization
- Problems with the use of semaphore: deadlock, starvation, and violation of mutual exclusion

Semaphores

The main disadvantage of the semaphore discussed in the previous section is that they all require **busy waiting**. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. This continual looping is clearly a problem in a real multiprogramming system, where a single CPU is shared among many processes. Busy waiting wastes CPU cycles that some other process may be able to use productively. This type of semaphore is also called a **spinlock** (because the process spins while waiting for the lock). Spinlocks are useful in multiprocessor systems. The advantage of a spinlock is that no context switch is required when a process must wait on a lock, and a context switch may take considerable time. This, when locks are expected to be held for short times, spinlocks are useful.

To overcome the need for busy waiting, we can modify the definition of semaphore and the wait and signal operations on it. When a process executes the wait operation and finds that the semaphore value is not positive, it must wait. However, rather than busy waiting, the process can block itself. The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state. Then, control is transferred to the CPU scheduler, which selects another process to execute.

A process that is blocked, waiting on a semaphore S, should be restarted when some other process executes a signal operation. The process is restarted by a wakeup operation, which changes the process from the waiting state to the ready state. The process is then placed in the ready queue. (The CPU may or may not be switched from the running process to the newly ready process, depending on the CPU scheduling algorithm.)

Such an implementation of a semaphore is as follows:

```
typedef struct {
  int value;
  struct process *L;
} semaphore;
```


CamScanner

executed, and statement S2 in P2 should execute only after statement S3 in P3 has executed. One possible semaphore-based solution uses two semaphores, A and B. Here is the solution.

```
semaphore A=0, B=0;
P1                  P2                  P3
...                 ...                 ...
wait(A);            wait(B);            S3;
S1;                 S2;                 signal(B);
                    signal(A);
...                 ...                 ...
```

Problems with Semaphores

Here are some key points about the use of semaphores:

- Semaphores provide a powerful tool for enforcing mutual exclusion and coordinating processes.
- The wait(S) and signal(S) operations are scattered among several processes. Hence, it is difficult to understand their effects.
- Usage of semaphores must be correct in all the processes.
- One bad (or malicious) process can fail the entire system of cooperating processes.

Incorrect use of semaphores can cause serious problems. We now discuss a few of these problems.

Deadlocks and Starvation

A set of processes are said to be in a deadlock state if every process is waiting for an event that can be caused only by another process in the set. Here are a couple of examples of deadlocks in our daily lives.

- Traffic deadlocks
- One-way bridge-crossing

Starvation is infinite blocking caused due to unavailability of resources. Here is an example of a deadlock.

```
P0                  P1
wait(S);            wait(Q);
wait(Q);            wait(S);
...                 ...
signal(S);          signal(Q);
signal(Q);          signal(S);
...                 ...
```

P0 and P1 need to get two semaphores, S and Q, before executing their critical sections. The following code structures can cause a deadlock involving P0 and P1. In this example, P0 grabs semaphore S and P1 obtains semaphore Q. Then, P0 waits for Q and P1 waits for S. P0 waits for P1 to execute signal(Q) and P1 waits for P0 to execute signal(S).

CamScanner

Operating Systems
Lecture No. 24

Reading Material

- Chapter 7 of the textbook
- Lecture 24 on Virtual TV

Summary

- Counting semaphores
- Classical synchronization problems
- Bounded buffer problem
- Readers and writers problem
- Dining philosophers problem

Semaphores

MOST Imp

There are two kinds of semaphores:

- **Counting semaphore** whose integer value can range over an unrestricted intege domain.
- **Binary semaphore** whose integer value cannot be > 1; can be simpler to implement

Let S be a counting semaphore. To implement it in terms of binary semaphores we need the following data structures:

```
binary-semaphore S1, S2;
int C;
```

Initially S1=1, S2=0, and the value of integer C is set to the initial value of the counting semaphore S. The wait operation on the counting semaphore S can be implemented as follows:

```
wait(S1);
C--;
if(C < 0) {
   signal(S1);
   wait(S2);
}
signal(S1);
```

The signal operation on the counting semaphore S can be implemented as follows:

```
wait(S1);
C++;
if(C <= 0)
  signal(S2);
else
  signal(S1);
```


CamScanner

Most Imp

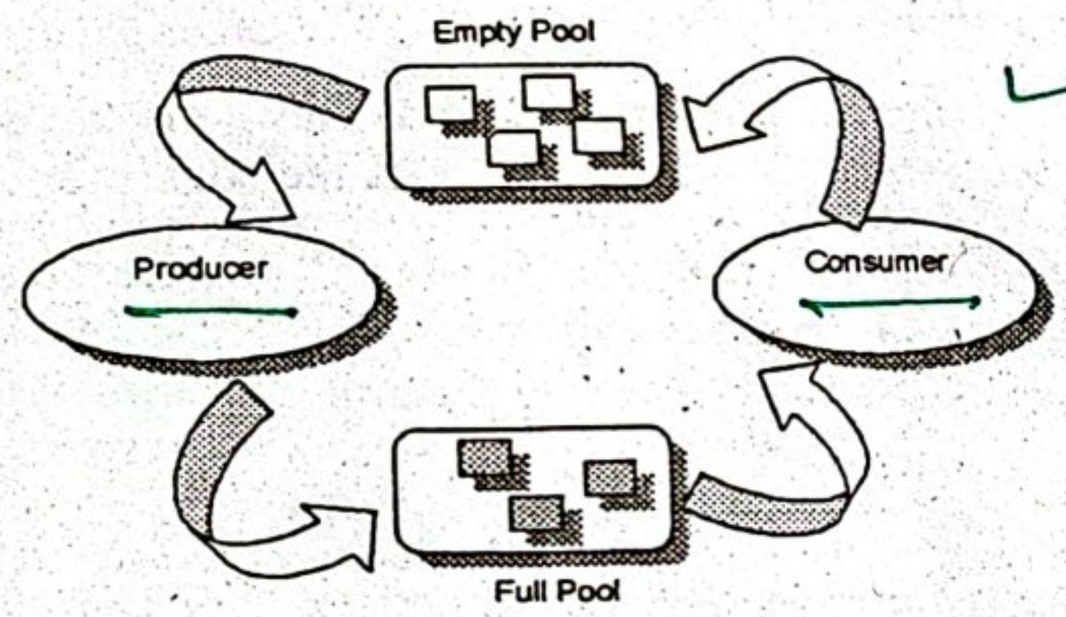
Classic Problems of Synchronization

The three classic problems of synchronization are:

- Bounded-Buffer Problem
- Readers and Writers Problem
- Dining Philosophers Problem

Bounded Buffer Problem

The bounded-buffer problem, which was introduced in a previous lecture, is commonly used to illustrate the power of synchronization primitives. The solution presented in this section assumes that the pool consists of n buffers, each capable of holding one item.



The mutex semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to the value 1. The empty and full semaphores count the number of empty and full buffers, respectively. The semaphore empty is initialized to the value n; the semaphore full is initialized to the value 0.

The code for the producer is as follows:

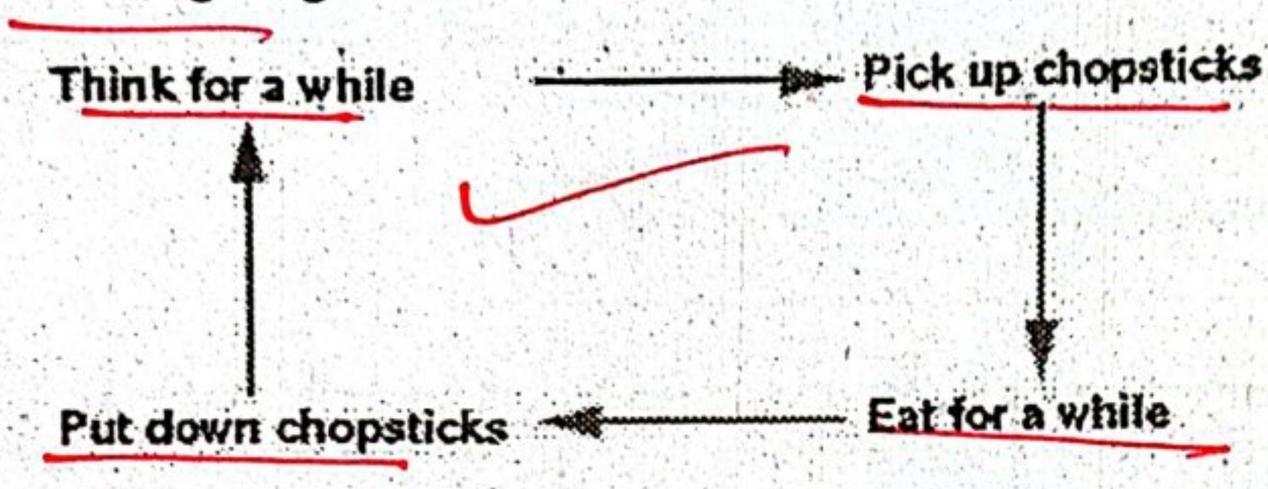
```
do {
    ...
    produce an item in nextp
    ...
    wait(empty);
    wait(mutex);
    ...
    add nextp to buffer
    ...
    signal(mutex);
    signal(full);
} while(1);
```

And that for the consumer is as follows:

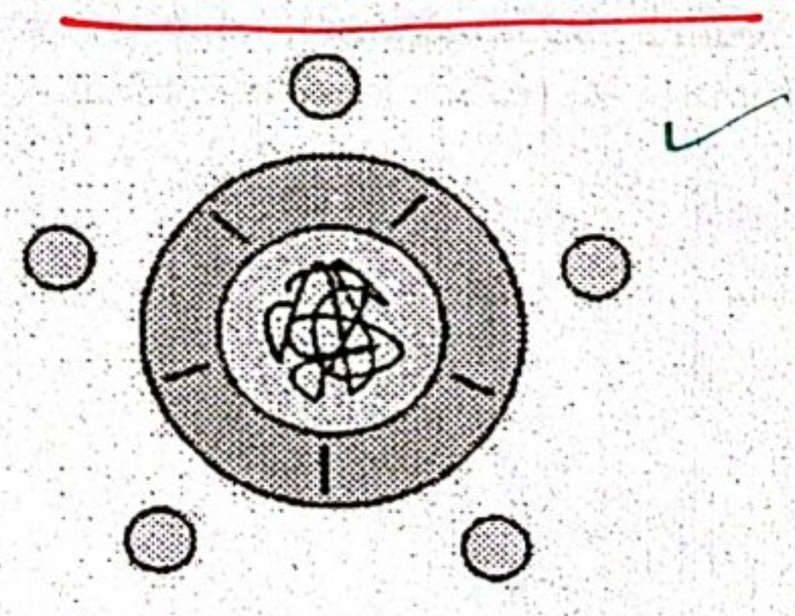
CamScanner

Dining Philosophers Problem

Consider five philosophers who spend their lives thinking and eating, as shown in the following diagram.



The philosophers share a common circular table surrounded by five chairs, each belonging to one philosopher. In the center of the table is a bowl of rice, and the table laid with five single chopsticks.



When a philosopher thinks, she does not interact with her colleagues. From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest her (the chopsticks that are between her and her left and right neighbors). A philosoph may pick up only one chopstick at a time. Obviously, she cannot pick up a chopstick t is already in the hand of her neighbor. When a hungry philosopher has both her chopsticks at the same time, she eats without releasing her chopsticks. When she is finished eating, she puts down both of her chopsticks and starts thinking again.

The dining philosophers problem is considered to be a classic synchronization problem because it is an example of a large class of concurrency control problems. It i simple representation of the need to allocate several resources among several processe a deadlock and starvation free manner.

One simple solution is to represent each chopstick by a semaphore. A philosopher tires to grab the chopstick by executing a wait operation on that semaphore; she releas her chopsticks by executing the signal operation on the appropriate semaphores. Thus shared data are:

```
semaphore chopstick[5];
```

All the chopsticks are initialized to 1. The structure of philosopher i is as follows:

CamScanner

```
struct buffer {
    item pool[n];
    int count,in,out;
};
```

The producer process inserts a new item (stored in nextp) into the shared buffer by executing

```
region buffer when(count < n) {
    pool[in] = nextp;
    in = (in+1)%n;
    count++;
}
```

The consumer process removes an item from the shared buffer and puts it in nextc by executing

```
region buffer when(count > 0) {
    nextc = pool[out];
    out = (out+1)%n;
    count--;
}
```

Monitors

Another high-level synchronization construct is the monitor type. A **monitor** is characterized by local data and a set of programmer-defined operators that can be used to access this data; local data be accessed only through these operators. The representation of a monitor type consists of declarations of variables whose values define the state of an instance of the type, as well as the bodies of procedures or functions that implement operations on the type. Normal scoping rules apply to parameters of a function and to its local variables. The syntax of the monitor is as follows:

```
monitor monitor_name
{
    shared variable declarations

    procedure body P1(..) { ...}
    procedure body P1(..) { ...}
    ...
    procedure body P1(..) { ...}
    {
        initialization code
    }
}
```

The monitor construct ensures that only one process at a time can be active within the monitor. Consequently, the programmer does not need to code this synchronization

CamScanner

```
dp.pickup(i);
    ...
    eat
    ...
dp.putdown(i);
```

It is easy to show that this solution ensures that no two neighbors are eating simultaneously and that no deadlocks will occur. We note, however, that it is possible for a philosopher to starve to death. You should think about this problem and satisfy yourself.

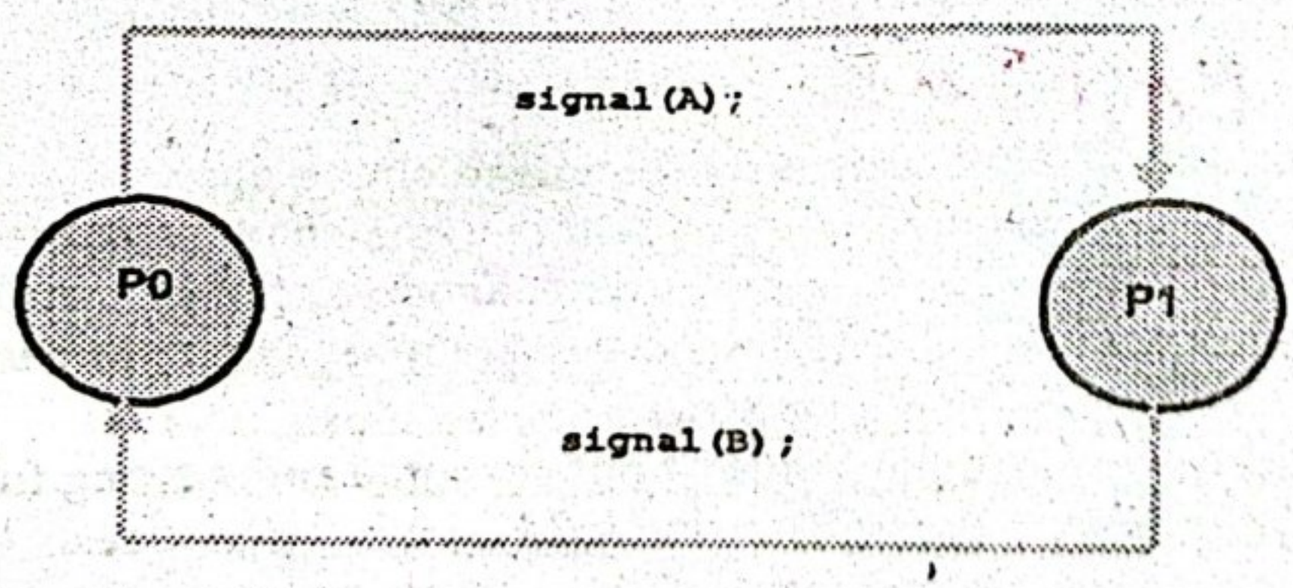
The Deadlock Problem

A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set. Here's an example:

- System has 2 tape drives.
- P1 and P2 each hold one tape drive and each needs another one.

Another deadlock situation can occur when the poor use of semaphores, as discussed in lecture 23. We reproduce that situation here. Assume that two processes, P0 and P1, need to access two semaphores, A and B, before executing their critical sections. Semaphores are initialized to 1 each. The following code snippets show how a situation can arise where P0 holds semaphore A, P1 holds semaphore B, and both wait for the other semaphore—a typical deadlock situation as shown in the figure that follows the code.

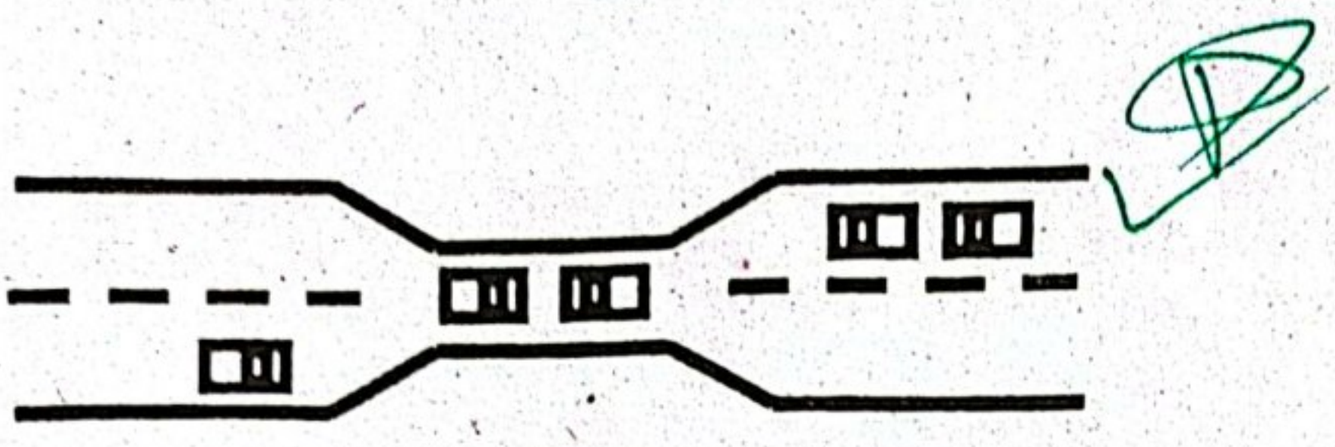
P0	P1
wait (A);	wait(B);
wait (B);	wait(A);



In the first solution for the dining philosophers problem, if all philosophers become hungry at the same time, they will pick up the chopsticks on their right and wait for getting the chopsticks on their left. This causes a deadlock.

Yet another example of a deadlock situation can occur on a one-way bridge, as shown below. Traffic flows only in one direction, and each section of a bridge can be viewed as a resource. If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback). Several cars may have to be backed up if a deadlock occurs. Starvation is possible.

CamScanner



In the next three to four lectures, we will discuss the issue of deadlocks in computer systems in detail.

System Model

A system consists of a finite number of resources to be distributed among a finite number of cooperating processes. The resources are partitioned into several types, each of which consists of some number of identical instances. Memory space, CPU cycles, disk drive, file are examples of resource types. A system with two identical tape drives is said to have two instances of the resource type disk drive.

If a process requests an instance of a resource type, the allocation of any instance of that type will satisfy the request. If it will not, then the instances are not identical and the resource type classes have not been defined properly.

A process must request a resource before using it, and must release the resource after using it. A process may request as many resources as it requires in order to carryout its designated task. Obviously, the number of resources requested may not exceed the total number of resources available in the system. Under the normal mode of operation, a process may utilize a resource in only the following sequence:

1. **Request:** The process requests a needed resource. If the request cannot be granted immediately, then the requesting process must wait until it can acquire the resource.
2. **Use:** The process can use the resource.
3. **Release:** The process releases the resource.

Deadlock Characterization

Most Imp

The following four conditions must hold simultaneously for a deadlock to occur:

1. **Mutual exclusion:** At least one resource must be held in a non-sharable mode; that is only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
2. **Hold and wait:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
3. **No preemption:** Resources cannot be preempted. That is, after using it a process releases a resource only voluntarily.
4. **Circular wait:** A set $\{P_0, P_1 \ldots P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, and so on, P_{n-1} is waiting for a resource held by P_n, and P_n is waiting for a resource held by P_0.

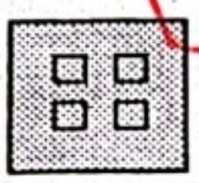
CamScanner

Resource Allocation Graphs

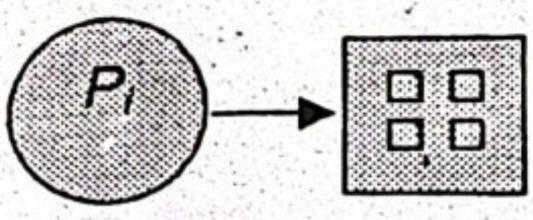
Deadlocks can be described more precisely in terms of a directed graph called a system **resource allocation graph**. This graph consists of a set of vertices V and a set of edges E. The set of vertices is portioned into two different types of nodes $P=\{P_0, P_1 \ldots P_n\}$, the set of the active processes in the system, and $R=\{R_0, R_1 \ldots R_n\}$, the set consisting of all resource types in the system. A directed edge from a process P_i to resource type R_j signifies that process P_i requested an instance of R_j and is waiting for that resource. A directed edge from R_j to P_i signifies that an instance of R_j has been allocated to P_i. We will use the following symbols in a resource allocation graph.

- Process

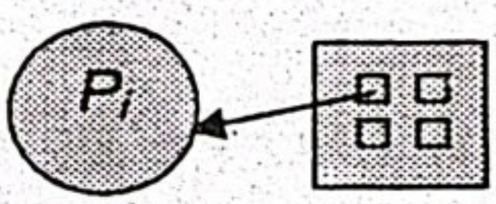
- Resource Type with 4 instances



- Pi requests instance of Rj



- Pi is holding an instance of Rj



The resource allocation graph shown below depicts the following situation:

- $P=\{P_1, P_2, P_3\}$
- $R=\{R_1, R_2, R_3\}$
- $E=\{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, P_3 \rightarrow R_3\}$

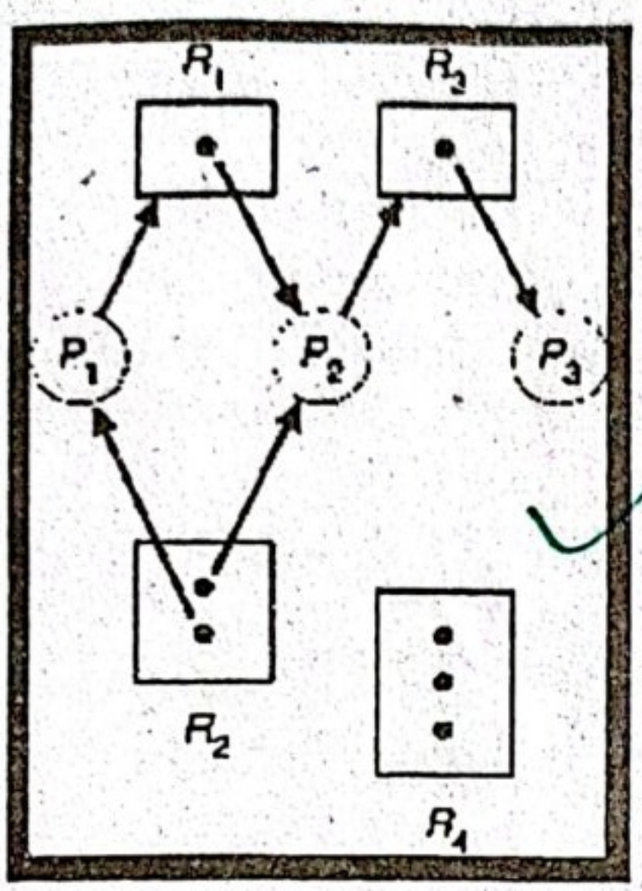
Resource Instances

- One instance of resource type R_1
- Two instances of resource type R_2
- One instance of resource type R_3
- Three instances of resource type R_4

Process States

- Process P_1 is holding an instance of resource R_2, and is waiting for an instance of resource R_1.
- Process P_2 is holding an instance of resource R_1 and R_2, and is waiting for an instance of resource R_3.
- Process P_3 is holding an instance of resource R_3.

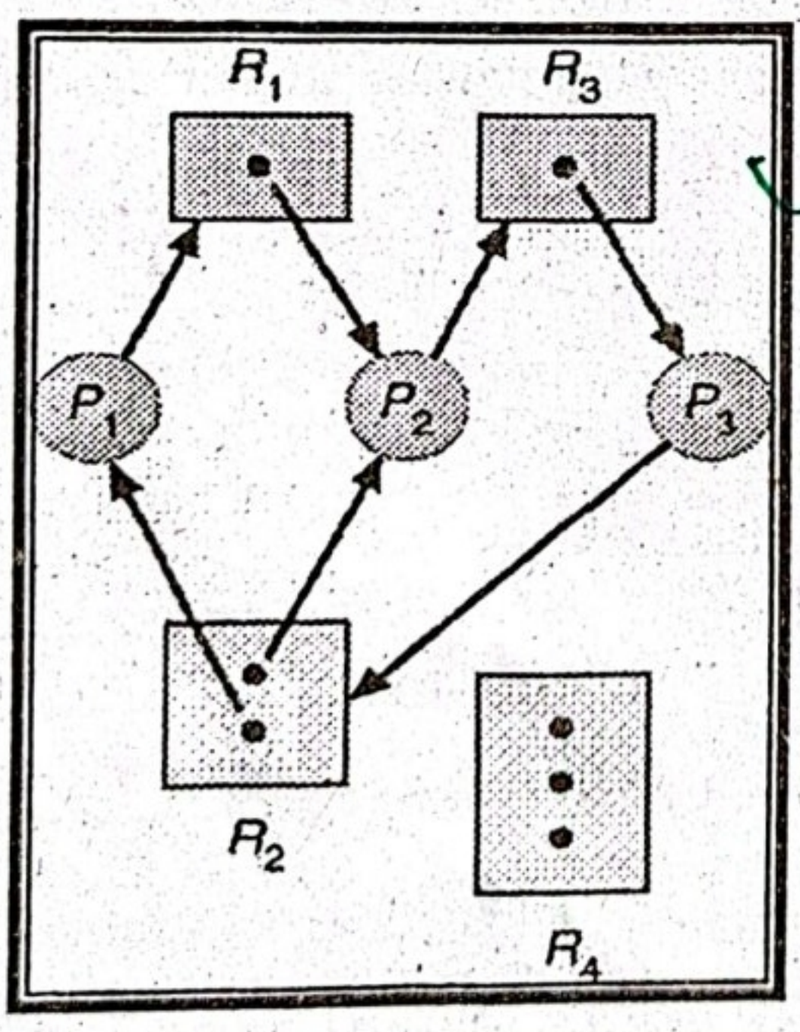
CamScanner



Given the definition of a resource allocation graph, it can be shown that if the graph contains no cycles, then no process is deadlocked. If the graph contains cycles then:

- If only one instance per resource type, then a deadlock exists.
- If several instances per resource type, possibility of deadlock exists.

Here is a resource allocation graph with a deadlock. There are two cycles in this graph:
$\{P_1 \rightarrow R_1, R_1 \rightarrow P_2, P_2 \rightarrow R_3, R_3 \rightarrow P_3, P_3 \rightarrow R_2, R_2 \rightarrow P_1\}$ and
$\{P_2 \rightarrow R_3, R_3 \rightarrow P_3, P_3 \rightarrow R_2, R_2 \rightarrow P_2\}$
No process will release an already acquired resource and the three processes will remain in the deadlock state.



CamScanner

Operating Systems
Lecture No. 27

Reading Material

- Chapter 8 of the textbook
- Lecture 27 on Virtual TV

Summary

- Deadlock handling
- Deadlock prevention
- Deadlock avoidance

Deadlock Handling

Three ways.

We can deal with deadlocks in a number of ways:

- Ensure that the system will never enter a deadlock state.
- Allow the system to enter a deadlock state and then recover from deadlock.
- Ignore the problem and pretend that deadlocks never occur in the system.

These three ways result in the following general methods of handling deadlocks:

1. **Deadlock prevention:** is a set of methods for ensuring that at least one of the necessary conditions cannot hold. These methods prevent deadlocks by constraining how processes can request for resources.
2. **Deadlock Avoidance:** This method of handling deadlocks requires that processes give advance additional information concerning which resources they will request and use during their lifetimes. With this information, it may be decided whether a process should wait or not.
3. **Allowing Deadlocks and Recovering:** One method is to allow the system to enter a deadlocked state, detect it, and recover.

Deadlock Prevention

MOST Imp

By ensuring that one of the four necessary conditions for a deadlock does not occur, we may prevent a deadlock.

Mutual exclusion

The mutual exclusion condition must hold for non-sharable resources, e.g., printer. Sharable resources do not require mutually exclusive access and thus cannot be involved in a deadlock, e.g., read-only files. Also, resources whose states can be saved and restored can be shared, such as a CPU. In general, we cannot prevent deadlocks by denying the mutual exclusion condition, as some resources are intrinsically non-sharable.

Hold and Wait

To ensure that the hold and wait condition does not occur in a system, we must guarantee that whenever a process requests a resource, it does not hold any other resources. One protocol that can be used requires each process to request and be allocated all its

CamScanner

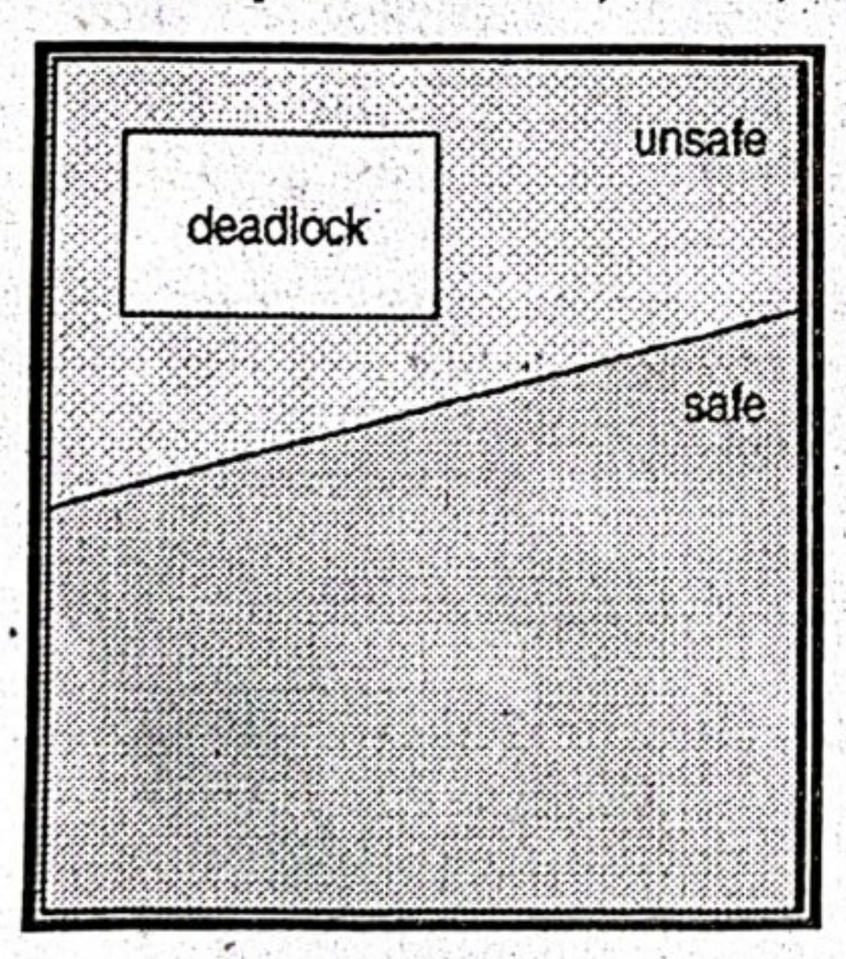
Deadlock Avoidance

One method for avoiding deadlocks is to require additional information about how resources may be requested. Each request for resources by a process requires that the system consider the resources currently available, the resources currently allocated to the process, and the future requests and releases of each process, to decide whether the current request can be satisfied or must wait to avoid a possible future deadlock. The simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need. Given a priori information about the maximum number of resources of each type that may be requested by each process, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state. A deadlock avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular wait condition can never exist.

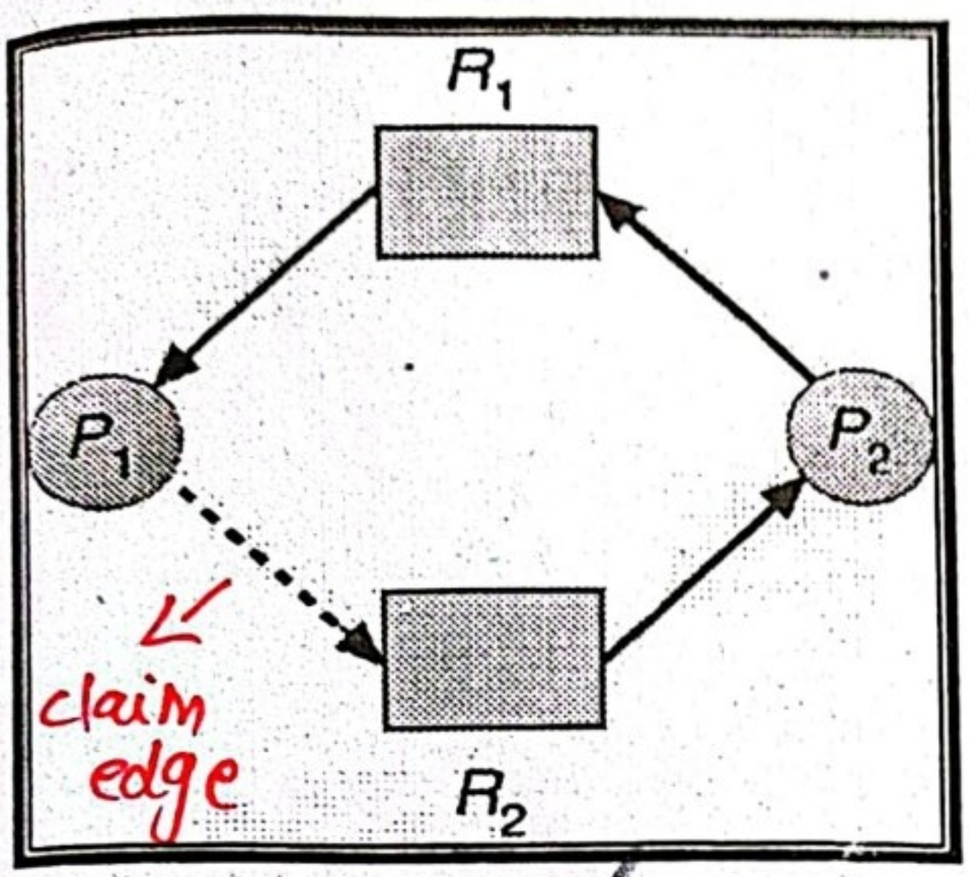
Safe State

A state is safe if the system can allocate resources to each process in some order and still avoid a deadlock. More formally a system is in a safe state only if there exists a safe sequence. A sequence of processes $<P_1, P_2 \ldots P_n>$ is a safe sequence for the current allocation state if, for each P_i, the resources that P_i can still request can be satisfied by the currently available resources plus all the resources held by all the P_j with $j < i$. In this situation, if the resources that P_i needs are not immediately available, then P_i can wait until all P_j have finished. When they have finished, P_i can obtain all of its needed resources, complete its designated task, return its allocated resources and terminate. When P_i terminates, P_{i+1} can obtain its needed resources and terminate. If no such sequence exists, then the system is said to be unsafe.

If a system is in a safe state, there can be no deadlocks. An unsafe state is not a deadlocked state; a deadlocked state is conversely an unsafe state. Not all unsafe states are deadlocks, however an unsafe state may lead to a deadlock state. Deadlock avoidance makes sure that a system never enters an unsafe state. The following diagram shows the relationship between safe, unsafe, and deadlock states.



CamScanner



Banker's Algorithm

In this algorithm, when a new process enters the system, it must declare the maximum number of instances of each resource type that it may need, i.e., each process must a priori claim maximum use of various system resources. This number may not exceed the total number of instances of resources in the system, and there can be multiple instances of resources. When a process requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise the process must wait until some other process releases enough resources. We say that a system is in a safe state if all of the processes in the system can be executed to termination in some order; the order of process termination is called safe sequence. When a process gets all its resources, it must use them and return them in a finite amount of time.

Let n be the number of processes in the system and m be the number of resource types. We need the following data structures in the Banker's algorithm:

- **Available:** A vector of length m indicates the number of available instances of resources of each type. Available[j] == k means that there are k available instances of resource R_j.
- **Max:** An n x m matrix defines the maximum demand of resources of each process. Max[i,j] == k means that process P_i may request at most k instances of resource R_j.
- **Allocation:** An n x m matrix defines the number of instances of resources of each type currently allocated to each process. Allocation[i,j] == k means that P_i is currently allocated k instances of resource type R_j.
- **Need:** An n x m matrix indicates the remaining resource need of each process. Need[i,j] == k means that P_i may need k more instances of resource type R_j to complete its task. Note that Need[i,j] == Max[i,j] - Allocation[i,j].

CamScanner

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1. Let Work and Finish be vectors of length m and n, respectively. Initialize Work = Available and Finish[i] = false fori = 1, 2, …, n.
2. Find an i such that both
   a) Finish[i] = = false
   b) $Need_i$ <= Work
   If no such i exists go to step 4.
3. Work = Work + $Allocation_i$
   Finish[i] = true
   Go to step 2
4. If Finish[i] = = true for all i, then the system is in a safe mode.

This algorithm may require an order of $m \times n^2$ operations to decide whether a state is safe.

Resource Request Algorithm

Let $Request_i$ be the request vector for process P_i. if $Request_i[j]=k$, then process P_i wants k instances of resource R_j. When a request for resources is made by process P_i the following actions are taken:

1. If $Request_i$ <= $Need_i$ go to step 2. Otherwise, raise an error condition since the process has exceeded its maximum claim.
2. If $Request_i$ <= Available, go to step 3. Otherwise P_i must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to process P_i by modifying the state as follows:

   Availabe = Available-$Request_i$;
   $Allocation_i$ = $Allocation_i$ + $Request_i$;
   $Need_i$ = $Need_i$ –$Request_i$;

Invoke the Safety algorithm. If the resulting resource allocation graph is safe, the transaction is completed. Else, the old resource allocation state is restored and process P_i must wait for $Request_i$.

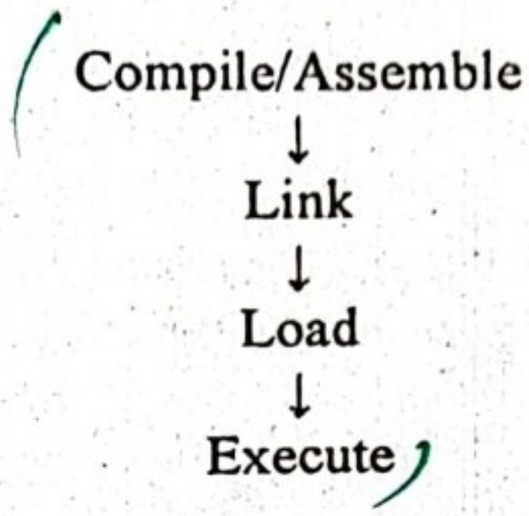
An illustrative example

We now show a few examples to illustrate how Banker's algorithm works. Consider a system with five processes P_0 through P_4 and three resource types: A, B, C. Resource type A has 10 instances, resource type B has 5 instances and resource type C has 7 instances. Suppose that at a time T_0 the following snapshot of the system has been taken:

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			

Source to Execution

Translation of a source program in a high-level or assembly language involves compilation and linking of the program. This process generates the machine language executable code (also known as a binary image) for the give source program. To execute the binary code, it is loaded into the main memory and the CPU state is set appropriately. The whole process is shown in the following diagram.



Address Binding

Usually a program resides on a disk as a binary executable or script file. The program must be brought into the memory it to be executed. The collection of processes that is waiting on the disk to be brought into the memory for execution forms the **input queue.**

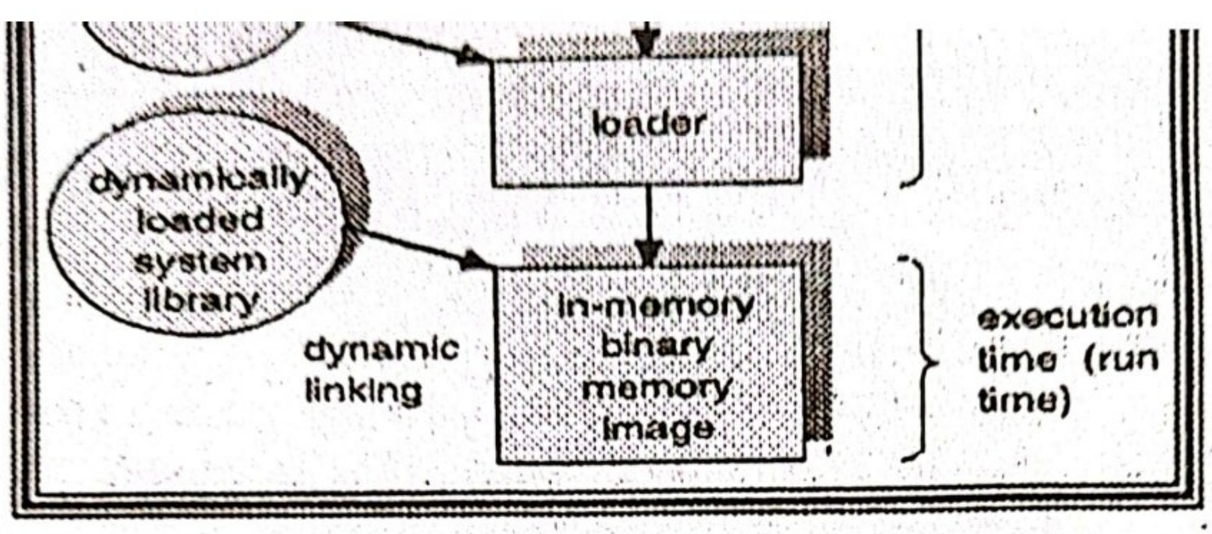
The normal procedure is to select one of the processes in the input queue and to load that process into the memory. As the process is executed, it accesses instructions and data from memory. Eventually the process terminates and its memory space is become available for reuse.

In most cases, a user program will go through several steps–some of which may be optional–before being executed. These steps are shown in the following diagram. Addresses may be bound in different ways during these-steps. Addresses in the source program are generally symbolic (such as an integer variable *count*). Address can be bound to instructions and data at different times, as discussed below briefly.

- **Compile time:** if you know at compile where the process will reside in memory, the **absolute addresses** can be assigned to instructions and data by the compiler.
- **Load time:** if it is not known at compile time where the process will reside in memory, then the compiler must generate **re-locatable code.** In this case the final binding is delayed until load time.
- **Execution time:** if the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time. Special hardware must be available for this to work.

In case of compile and load time binding, a program may not be moved around in memory at run time.

CamScanner



Logical- Versus Physical-Address Space

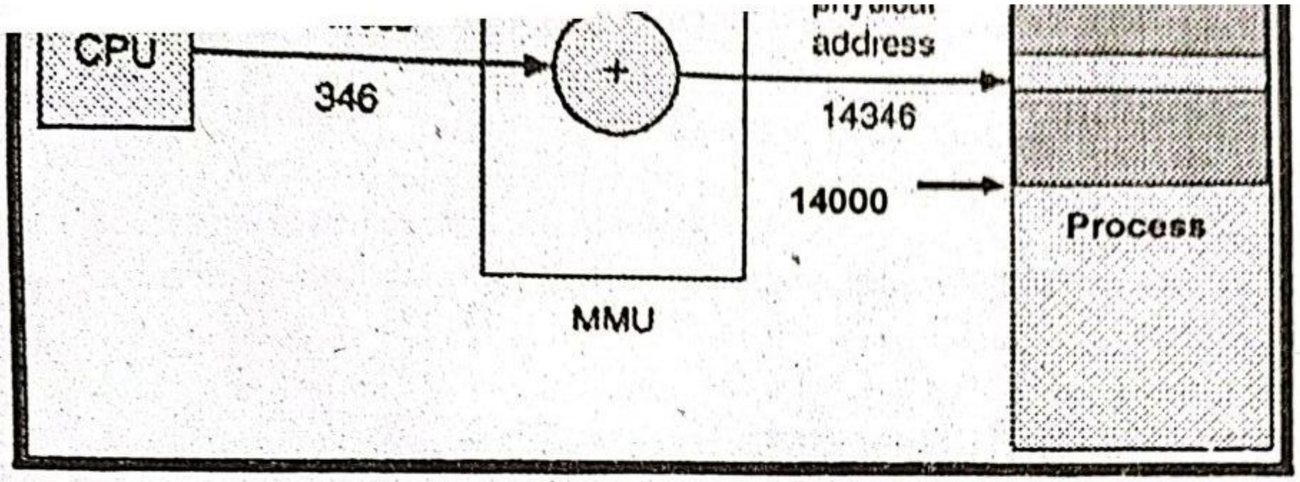
An address generated by the CPU is commonly referred to as a **logical address,** where as an address seen by the memory unit–that is, the one loaded into the **memory-address register** of the memory–is commonly referred to as the **physical address.** In essence, logical data refers to an instruction or data in the process address space where as the physical address refers to a main memory location where instruction or data resides.

The compile time and load time binding methods generate identical logical and physical addresses, where as the execution time binding method results in different physical and logical addresses. In this case we refer to the logical address as the **virtual address.** The set of all logical addresses generated by a program form the **logical address space** of a process; the set of all physical addresses corresponding to these logical addresses is a **physical address space** of the process. The total size of physical address space in a system is equal to the size of its main memory.

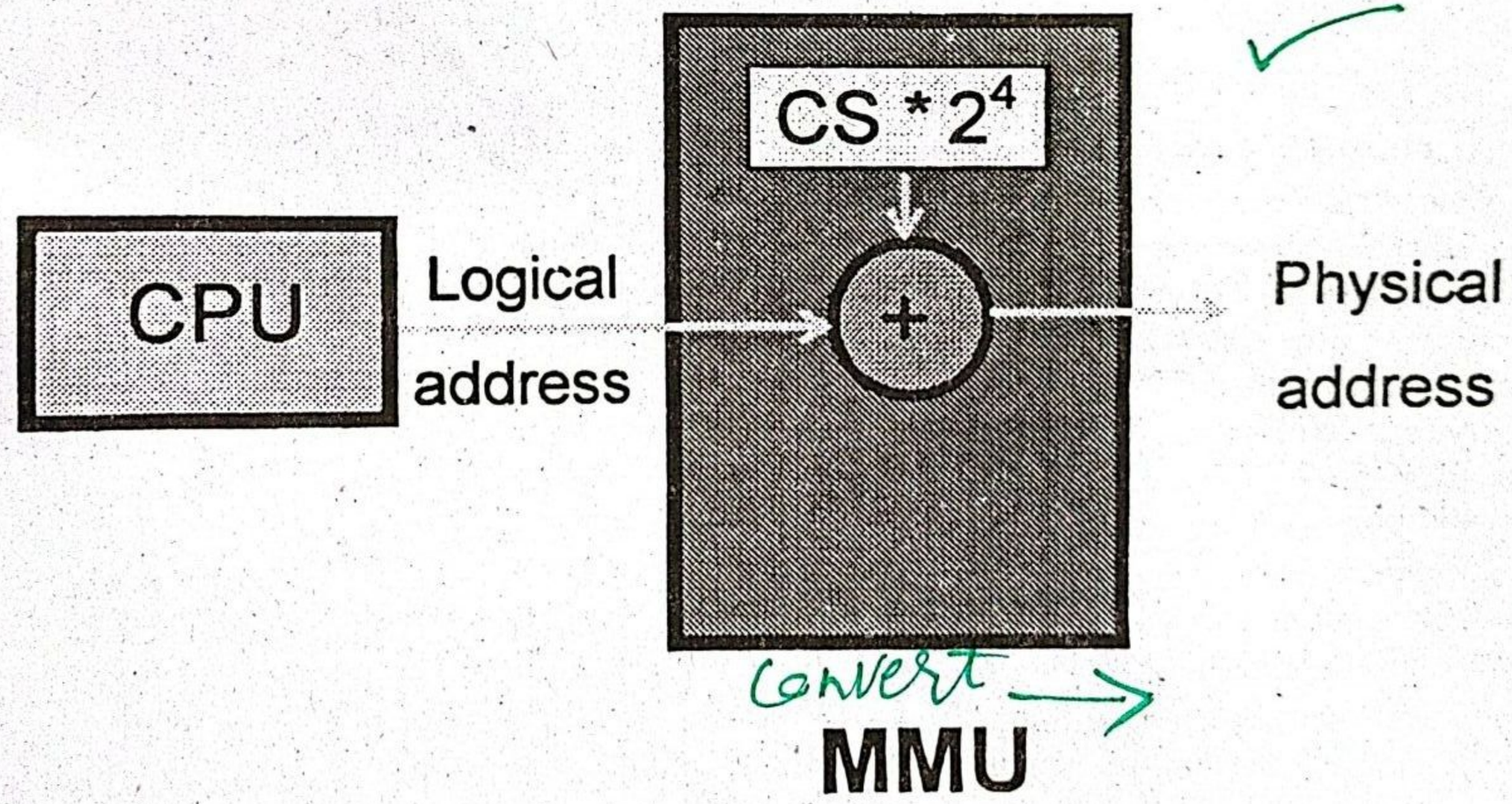
The run-time mapping from virtual to physical addresses is done by a piece of hardware in the CPU, called the **memory management unit (MMU).**

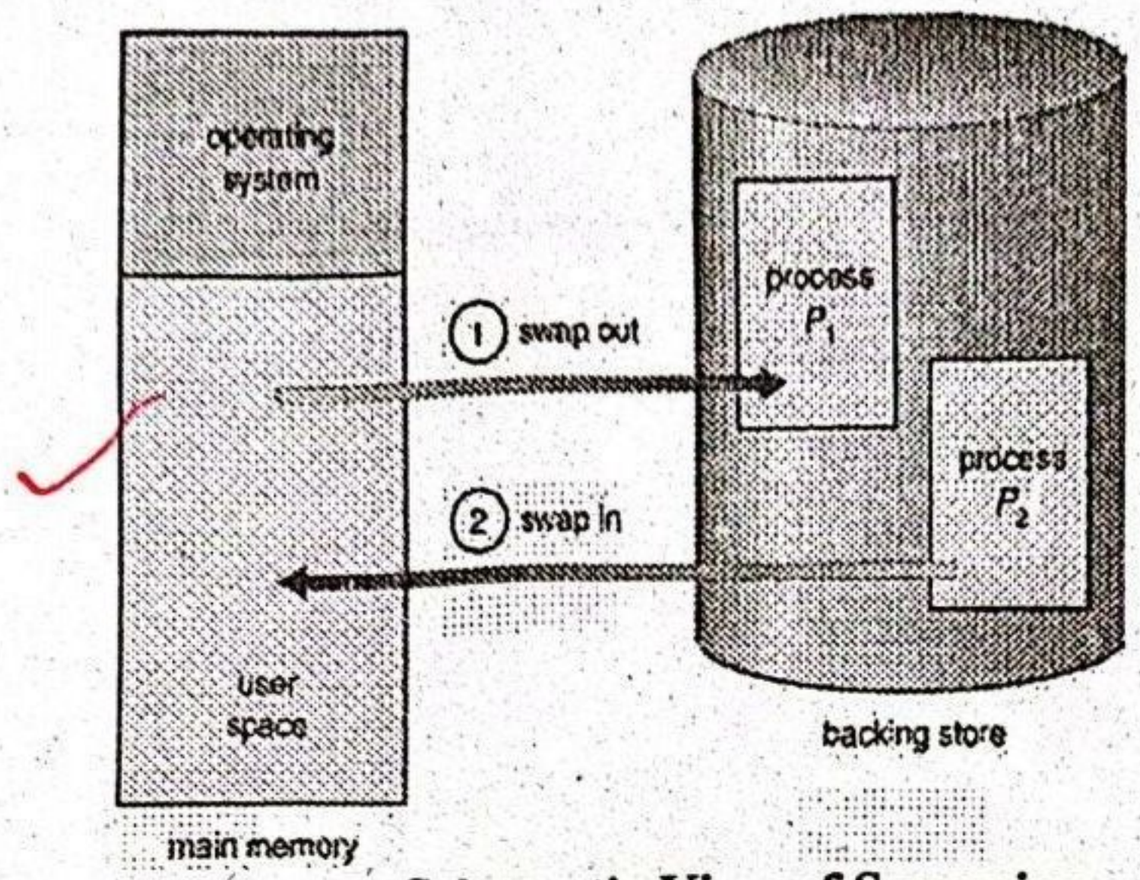
CamScanner

CamScanner



In i8086, the logical address of the next instruction is specified by the value of **instruction pointer** (IP). The physical address for the instruction is computed by shifting the **code segment register** (CS) left by four bits and adding IP to it, as shown below.





Schematic View of Swapping

Cost of Swapping

Process size	= 1 MB
Transfer rate	= 5 MB/sec
Swap out time	= 1/5 sec
	= 200 ms
Average latency	= 8 ms
Net swap out time	= 208 ms
Swap out + swap in	= 416 ms

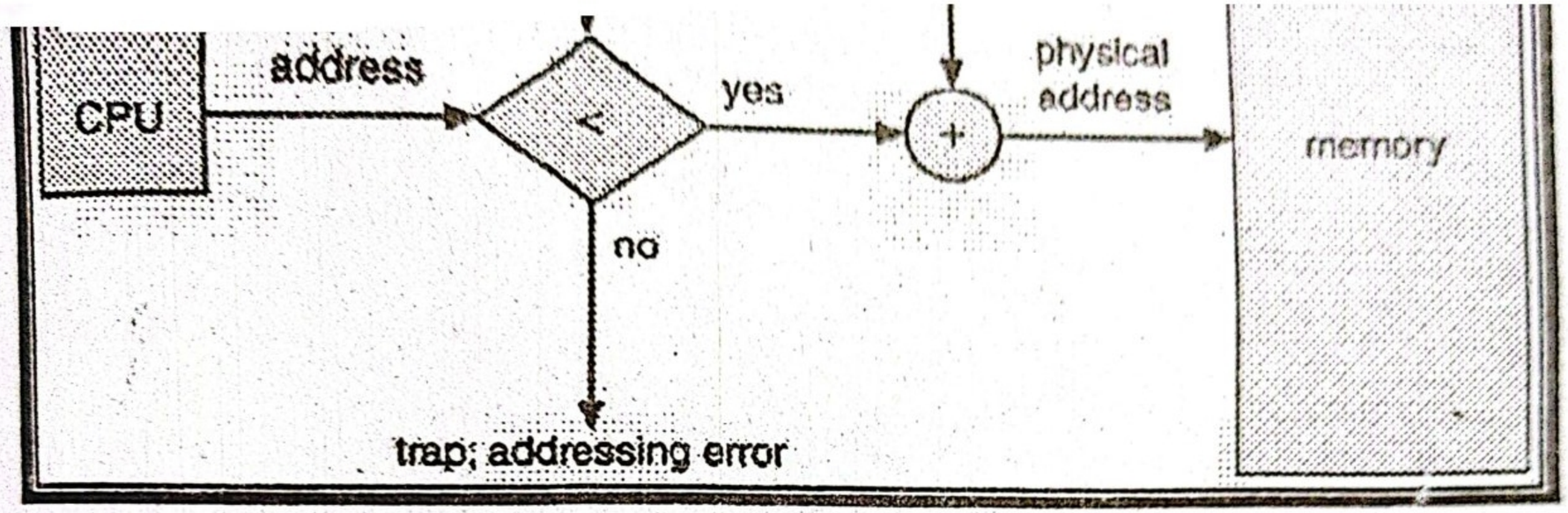
Contiguous memory allocation

The main memory must accommodate both operating system and the various user spaces. Thus memory allocation should be done efficiently.

The memory is usually divided into two partitions: one for the resident operating system and one for the user processes. The operating system may be placed in the high memory or the low memory. The position of the interrupt vector usually affects this decision. Since the interrupt vector is often in the low memory, programmers place the OS in low memory too.

- It is desirable to have several user processes residing in the memory at the same time. In contiguous memory allocation, each process is contained in a single contiguous section of memory. The **base** (re-location) and **limit** registers are used to point to the smallest memory address of a process and its size, respectively.

CamScanner



Contiguous Allocation

Multiprogramming with Fixed Tasks (MFT)

In this technique, memory is divided into several fixed-size partitions. Each partition may contain exactly one process. Thus the degree of multiprogramming is bound by the number of partitions. In this multiple partition method, when a partition is free, a process is selected from the input queue and is loaded in the free partition. When the process terminates, the partition becomes available for another process.

- This was used by IBM for system 360 OS/MFT (multiprogramming with a fixed number of tasks).
- Can have a single input queue instead of one for each partition.
  - So that if there are no big jobs can use big partition for little jobs.
  - Can think of the input queue(s) as the ready list(s) with a scheduling policy of

CamScanner

Operating Systems
Lecture No. 32

Reading Material

- Chapter 9 of the textbook
- Lecture 32 on Virtual TV

Summary

- MVT
- Paging
- Logical to physical address translation

Multiprogramming with Variable Tasks (MVT)

This is the generalization of the fixed partition scheme. It is used primarily in a batch environment. This scheme of memory management was first introduced in IBM OS/MVT ... with a varying number of tasks). Here are the main characteristics of

Operating Systems
Lecture No. 33

Reading Material

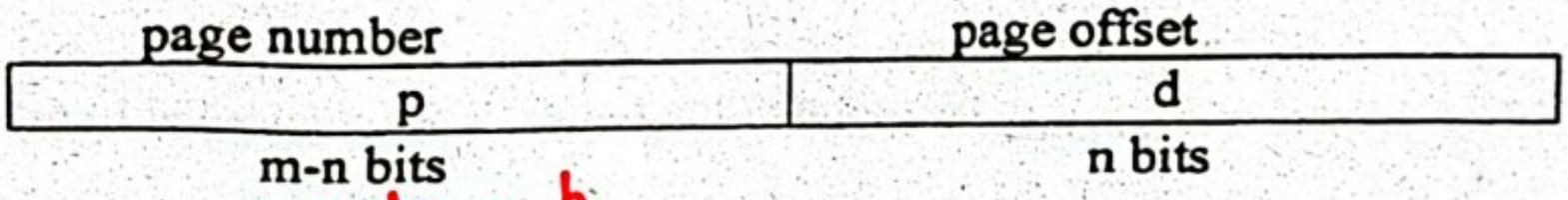
- Chapter 9 of the textbook
- Lecture 33 on Virtual TV

Summary

- Addressing and logical to physical address translation
- Examples: Intel P4 and PDP-11
- Page table implementation
- Performance of paging

Addressing in Paging

The page size is defined by the CPU hardware. If the size of logical address space is 2^m and a page size is 2^n addressing units (bytes or words) , then the high-order *m-n* bits of a logical address designate the page number and the *n* low order bits designate offset within the page. Thus, the logical address is as follows:



Example:

Most Imp

Assume a **logical address space** of 16 pages of 1024 words, each mapped into a physical memory of 32 frames. Here is how you calculate the various parameters related to paging.

No. of bits needed for **p** = ceiling [$\log_2$ 16] bits = 4 bits
No. of bits neede for **f** = ceiling [$\log_2$ 32] bits = 5 bits
No. of bits needed for **d** = ceiling [$\log_2$ 2048] bits = 11 bits
Logical address size = |p| + |d| = 4+11 bits = 15 bits
Physical address size = |f| + |d| = 5+11 bits = 16 bits

Page Table Size

Page table size= NP * PTES , where NP is the number of pages in the process address space and PTES is the page table entry size (equal to |f| based on our discussion so far).

Page table size = 16 * 5 bits (for the above example; assuming a byte size page table entry)

Paging in Intel P4

32-bit *linear* address

CamScanner

Operating Systems
Lecture No. 34

Reading Material

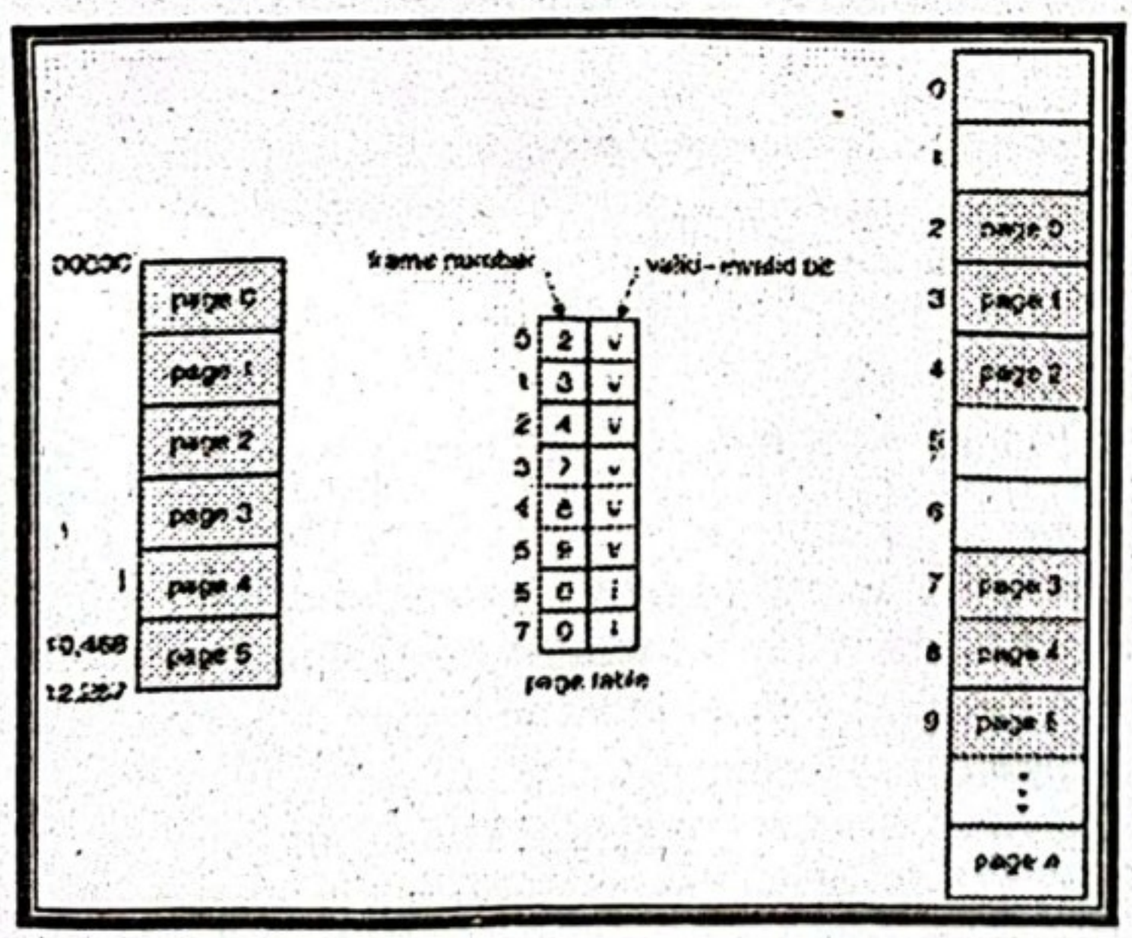
- Chapter 9 of the textbook
- Lecture 34 on Virtual TV

Summary

- Protection under paging
- Structure of the page table
  - Multi-level paging
  - Hashed page tables
  - Inverted page table

Protection under Paging

Memory protection in paging is achieved by associating protection bits with each page. These bits are associated with each page table entry and specify protection on the corresponding page. The primary protection scheme guards against a process trying to access a page that does not belong to its address space. This is achieved by using a valid/invalid (v) bit. This bit indicates whether the page is in the process address space or not. If the bit is set to invalid, it indicates that the page is not in the process's logical address space. Illegal addresses are trapped by using the valid-invalid bit and control is passed to the operating system for appropriate action. The following diagram shows the use of v bit in the page table. In this case, logical address space is six page and any access to pages 6 and 7 will be trapped because the v bits for these pages is set to invalid.



Use of valid/invalid (v) bit for protection under paging

One bit can define the page table to be read and write or read only. Every reference to memory goes through the page table to find the correct frame number. At the same time that the physical address is being computed, the protection bits can be checked to verify that no writes are being made to a read only page. An attempt to write to a read-only page causes a hardware trap to the operating system (memory-protection violation).

This approach can be expanded to provide a finer level of protection. Read, write, and execute bits (r, w, x) can be used to allow a combination of these accesses, similar to the file protection scheme used in the UNIX operating system. Illegal attempts will be trapped to the operating system.

Structure of the Page Table

As logical address spaces become large (32-bit or 64-bit), depending on the page size, page table sizes can become larger than a page and it becomes necessary to page the page the page table. Additionally, large amount of memory space is used for page table. The following schemes allow efficient implementations of page tables.

- Hierarchical / Multilevel Paging
- Hashed Page Table
- Inverted Page Table

Hierarchical/Multilevel Paging

Most modern computers support a large logical address space: (2^{32} to 2^{64}). In such an environment, the page table itself becomes excessively large. Consider the following example:

- Logical address = 32-bit
- Page size = 4K bytes (212 bytes)
- Page table entry = 4 bytes
- Maximum pages in a process address space = 232 / 4K = 1M
- Maximum pages in a process address space = 232 / 4K = 1M
- Page table size = 4M bytes

This page table cannot fit in one page. One solution is to page the page table, resulting in a 2-level paging. A page table needed for keeping track of pages of the page table—called the outer page table or page directory. In the above example:

- No. of pages in the page table is 4M / 4K = 1K
- Size of the outer page table is 1K * 4 bytes = 4K bytes ⇒ outer page will fit in one page

In the 32-bit machine described above, we need to partition p into two parts, p1 and p2. p1 is used to index the outer page table and p2 to index the inner page table. Thus the logical address is divided into a page number consisting of 20 bits and a page offset of 12 bits. Since we page the page table, the page number is further divided into a 10-bit page number, and a 10-bit page offset. This is known as **two-level paging**. The following diagram shows division of the logical address in 2-level paging and hierarchical views of the page table.

CamScanner

- Size of the outer page table (2K * 4 = 8 KB) is further paged, resulting in 3-level paging per section

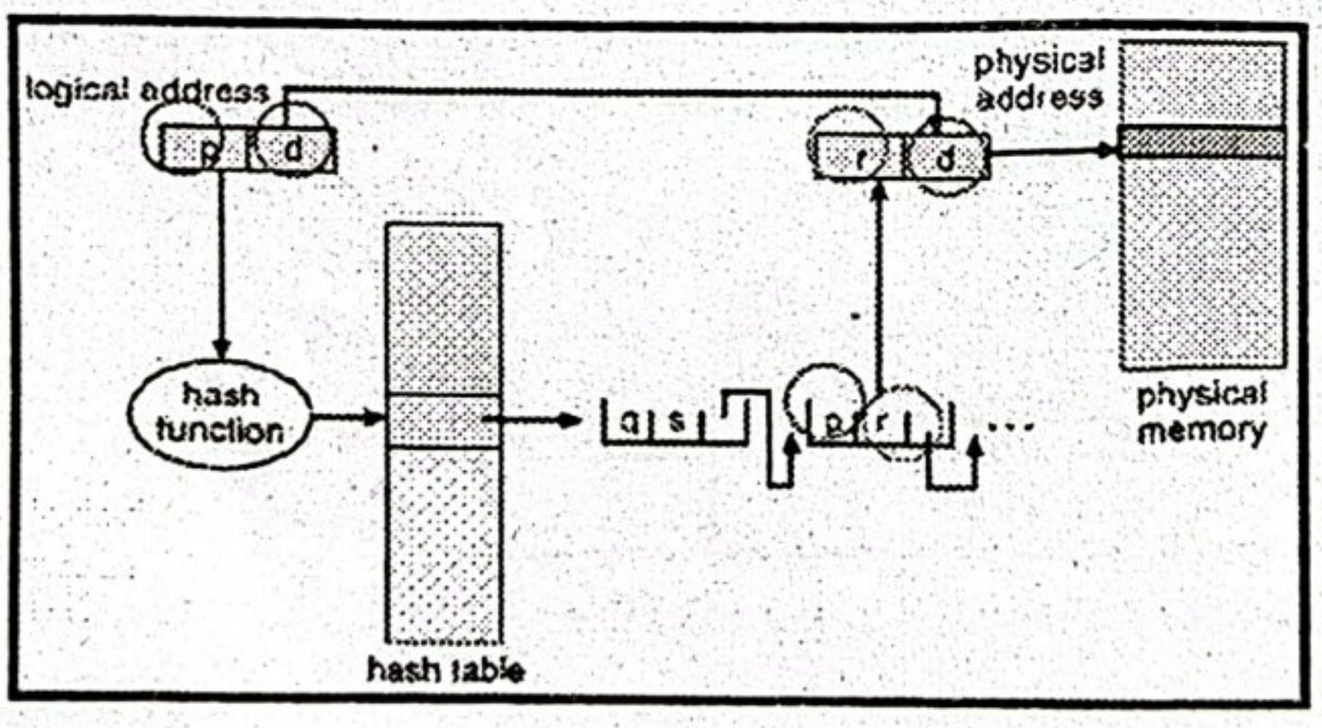
Section	Page number	Page offset
s	p	d
2	21	9

More Examples

- 32-bit Sun SPARC supports 3-level paging
- 32-bit Motorola 68030 supports 4-level paging
- 64-bit Sun UltraSPARC supports 7-level paging – too many memory references needed for address translation

Hashed Page Table

This is a common approach to handle address spaces larger then 32 bits .Usually open hashing is used. Each entry in the linked list has three fields: page number, frame number for the page, and pointer to the next element—(p, f, next). The page number in the logical address (specified by p) is hashed to get index of an entry in the hash table. This index is used to search the linked list associated with this entry to locate the frame number corresponding to the given page number. The advantage of hashed page tables is smaller page tables.



Inverted Page Table

Usually each process has a page table associated with it. The page table has one entry for each page in the address space of the process. For large address spaces (32-bit and above), each page table may consist of millions of entries. These tables may consume large amounts of physical memory, which is required just to keep track of how the mapping of logical address spaces of processes onto the physical memory.

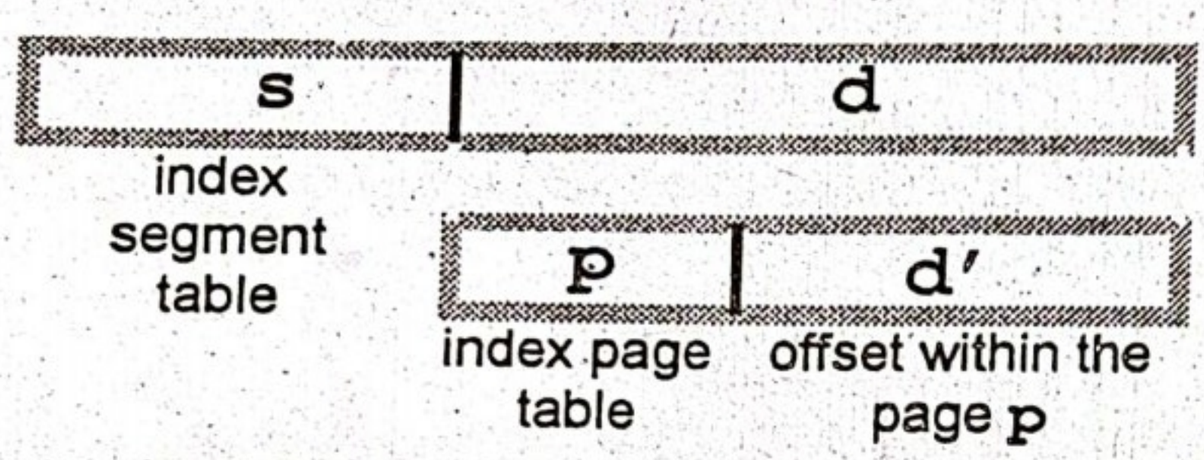
A solution is to use an inverted page table. An **inverted page table** has one entry for each real page (frame) of memory. Each entry consists of the virtual address of the page stored in the in that real memory location, with information about the process that own the page.

CamScanner

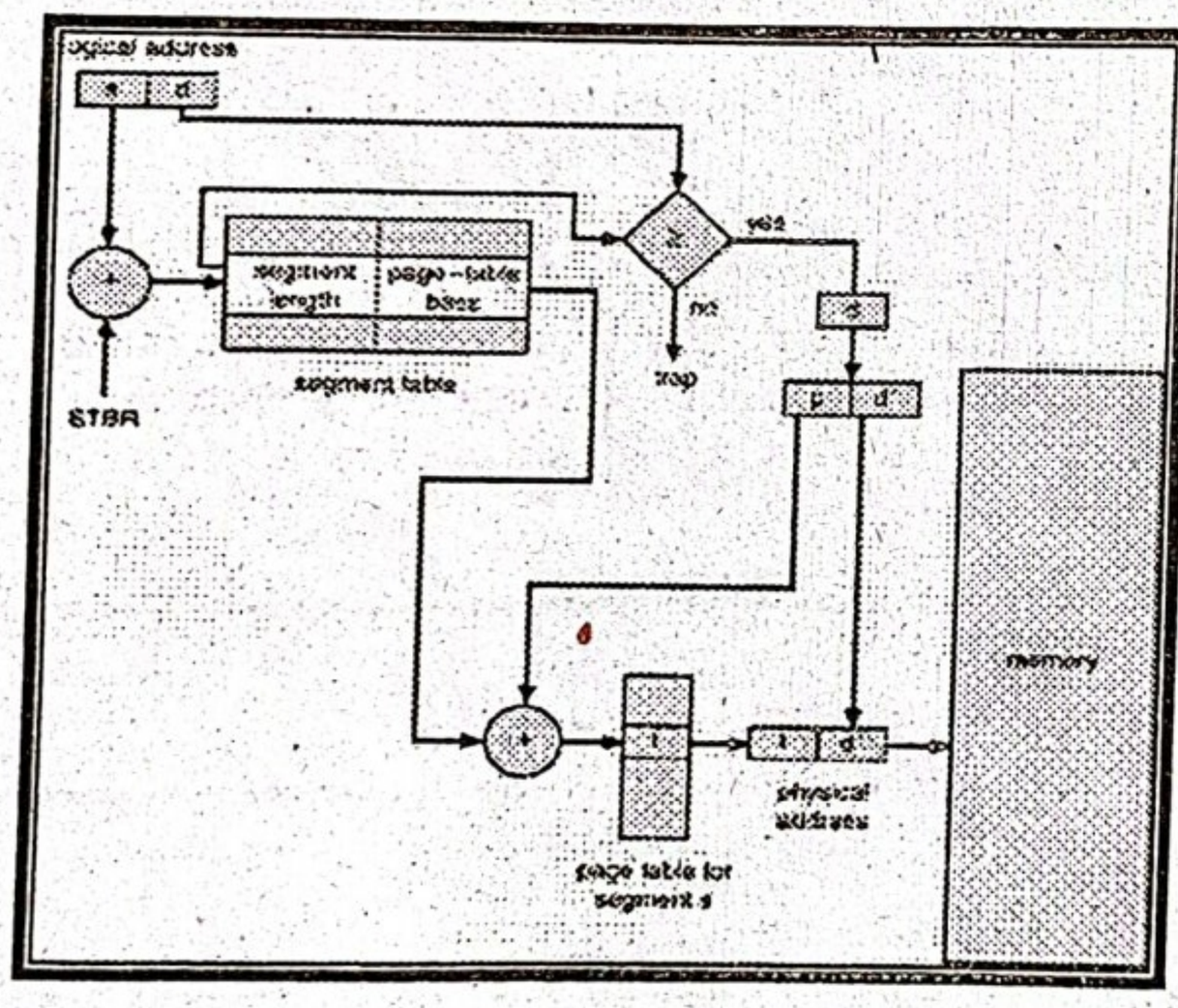
The logical address is still <s,d>, with s used to index the segment table. Each segment table entry consist of the tuple

<segment-length, page-table-base>

The logical address is legal if d < segment-length. The segment offset, d, is partitioned into two parts: p and d', where p is used to index the page table associated with the segment s and d' is used as offset within the page p. p indexes the page table to retrieve frame, f, and physical address (f,d') is formed. The following diagrams show the format of logical address and its division, and the hardware support needed for logical to physical address translation.



a) Logical address and its partition



b) Hardware support needed for logical to physical address translation

MULTICS Example

We now take the example of one of the finest operating systems of late 1960s and early 1970s, known as the MULTICS operating system. Here are the specifications of the CPU supported by MULTICS and calculation of its various parameters such as the largest segment size supported by MULTICS.

- GE 345 processor
- Logical address = 34 bits

7. Interrupt from the disk (I/O completed)
8. Save the registers and process state for the other user (if step 6 is executed)
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show that the desired page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state and new page table

Not all these steps are necessary in every case. For example we are assuming that in step 6, the CPU is allocated to another process while the I/O occurs. This arrangement allows multiprogramming to maintain CPU utilization, but requires additional time to resume the page fault service routine when the I/O transfer is complete.

In any case we are faced with three major components of the page fault service time:

1. Service the page fault interrupt
2. Read in the page
3. Restart the process

The first and third tasks may be reduced, with careful coding, to several hundred instructions. These tasks may take from 1 to 100 microseconds each. The page switch time, on the other hand, will probably be close to 24 milliseconds. A typical hard disk has an average latency of 8 milliseconds, a seek of 15 milliseconds, and a transfer time of 1 millisecond. Thus, the total paging time would be close to 25 milliseconds, including hardware and software time. Remember that we are looking at only the device service time. If a queue of processes is waiting for the device we have to add device queuing time as we wait for the paging device to be free to service our request, increasing even more the time to swap.

If we take an average page fault service time of 25 milliseconds and a memory access time of 100 nanoseconds, then the effective access time in nanoseconds is

$$\begin{aligned}\text{Effective access time} &= (1-p) * (100) + p\ (25 \text{ milliseconds}) \\ &= (1-p) * 100 + p * 25{,}000{,}000 \\ &= 100 + 24{,}999{,}900 * p\end{aligned}$$

We see then that the effective access time is directly proportional to the page fault rate. If one access out of 1,000 causes a page fault, the effective access time is 25 microseconds. The computer would be slowed down by a factor of 250 because of demand paging! If we want less than 10 percent degradation, we need:

$$\begin{aligned}110 &> 100 + 25{,}000{,}000 * p \\ 10 &> 25{,}000{,}000 * p \\ p &< 0.0000004\end{aligned}$$

That is, to keep the slowdown due to paging to a reasonable level, we can allow only less than one memory access out of 2,500,000 to page fault.

It is important to keep the slowdown due to paging to a reasonable level, we can allow only less than one memory access out of 2,500,000 to page fault.

It is important to keep the page fault rate low in a demand-paging system. Otherwise the effective access time increases, slowing process execution dramatically.

One additional aspect of demand paging is the handling and overall use of swap space. Disk I/O to swap space is generally faster than that to the file system. It is faster because swap space is allocated in much larger blocks, and file lookups and indirect allocation methods are not used. It is therefore possible for the system to gain better paging throughput by copying an entire file image into the swap space at process startup,

CamScanner

Operating Systems
Lecture No. 39

39 & 40 Imp lectures.

Reading Material

- Chapter 10 of the textbook
- Lecture 39 on Virtual TV

Summary

- Page replacement (basic concept and replacement algorithms)

Page replacement

While a user process is executing, a page fault occurs. The hardware traps to the operating system, which checks its internal tables to see that this page is a genuine one rather than an illegal memory access. The operating system determines where the desired page is residing on the disk, but then finds that there are no free frames on the free frame list: All memory is in use.

The operating system has several options at his point. It could terminate the user process. However, demand paging is the operating system's attempt to improve the computer system's utilization and throughput. Users' should not be aware that their processes are running on a paged system – paging should be logically transparent to the user. So this option is not the best choice. The operating system could swap out a process, but that would reduce the level of multiprogramming. So we explore page replacement. This means that if no free frame is available on a page fault, we replace a page in memory to load the desired page. The page-fault service routine is modified to include page replacement. We can free a frame by writing its contents to swap space, and changing the page table to indicate that the page is no longer in memory. The modified page fault service routine is:

1. Find the location of the desired page on the disk
2. Find a free frame
   a) If there is a free frame use it.
   b) If there is no free frame, use a page replacement algorithm to select a victim frame.
3. Read the desired page into the newly freed frame; change the page and frame tables.
4. Restart the user process.

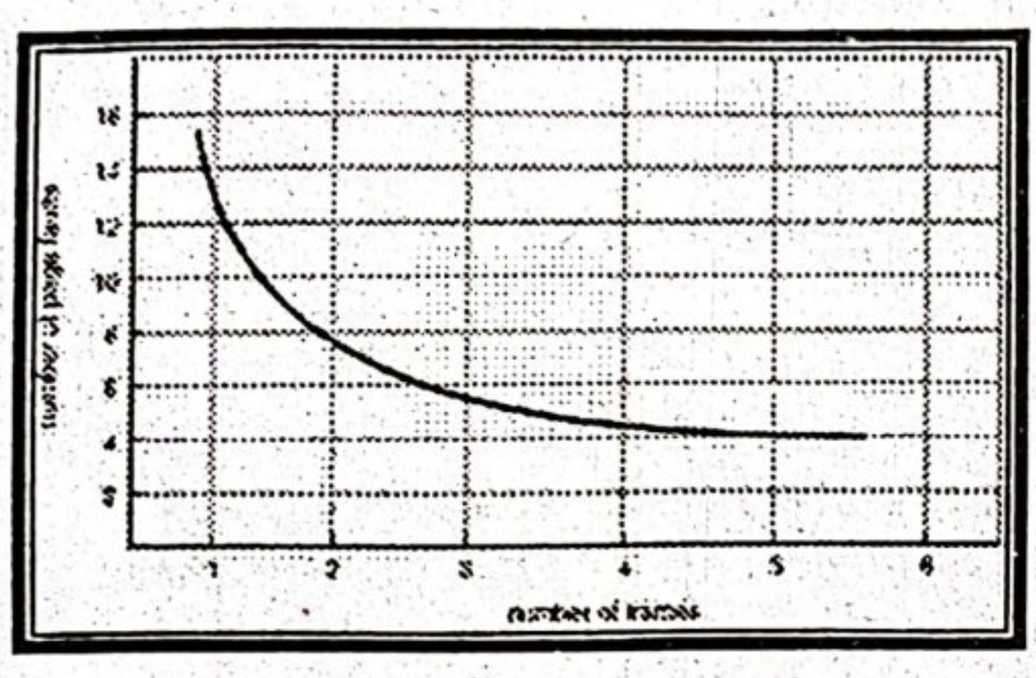
The following diagram shows theses steps pictorially.

CamScanner

Page Replacement Algorithms

In general we want a page replacement algorithm with the lowest page-fault rate. We evaluate an algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string.

To determine the number of page faults for a particular reference string and page replacement algorithm, we also need to know the number of page frames available. Obviously as the number of frames available increases, the number of page faults decreases.



Expected relationship between number of free frames allocated to a process and the number of page faults caused by it

FIFO Page Replacement

The simplest page-replacement algorithm is a FIFO algorithm. A FIFO replacement algorithm associates with each page the time when that page was brought into memory. When a page must be replaced, the oldest page is chosen. Notice that it is not strictly necessary to record the time when a page is brought in. We can create a FIFO queue to hold all pages in memory. We replace the page at the head of the queue. When a page is brought into memory we insert t at the tail of the queue.

Example: Consider the following example, in which the number of frames allocated is 4, and the reference string is 1, 2, 3, 4, 5, 1, 6, 7, 8, 7, 8, 9, 5, 4, 5, 4, 4. The number of page faults caused by the process is nine, as shown below.

1, 2, 3, 4, 5, 3, 4, 1, 6, 7, 8, 7, 8, 9, 5, 4, 5, 4, 4

1	5	5	5	5	8	8	8	8
2	2	1	1	1	1	9	9	9
3	3	3	6	6	6	6	5	5
4	4	4	4	7	7	7	7	4

Example for the FIFO page replacement algorithm

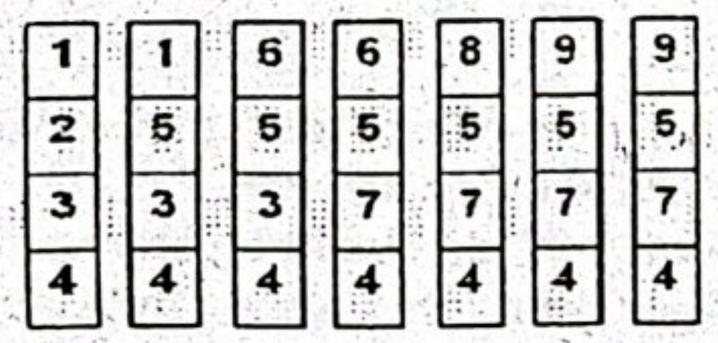
The problem with this algorithm is that it suffers from Belady's anomaly: For some page replacement algorithms the page fault rate may increase as the number of allocated

frames increases, whereas we would expect that giving more memory to a process would improve its performance.

Optimal Algorithm

An optimal page-replacement algorithm has the lowest page fault rate of all algorithms, and will never suffer from the Belay's algorithm. This algorithm is simply to replace the page that will not be used for the longest period of time. Use of this algorithm guarantees the lowest possible page-fault rate for a fixed number of frames. In case of the following example (which uses the same replacement string as the example for the FIFO algorithm), the number of page faults caused by the process is seven.

1, 2, 3, 4, 5, 3, 4, 1, 6, 7, 8, 7, 8, 9, 5, 4, 5, 4, 4

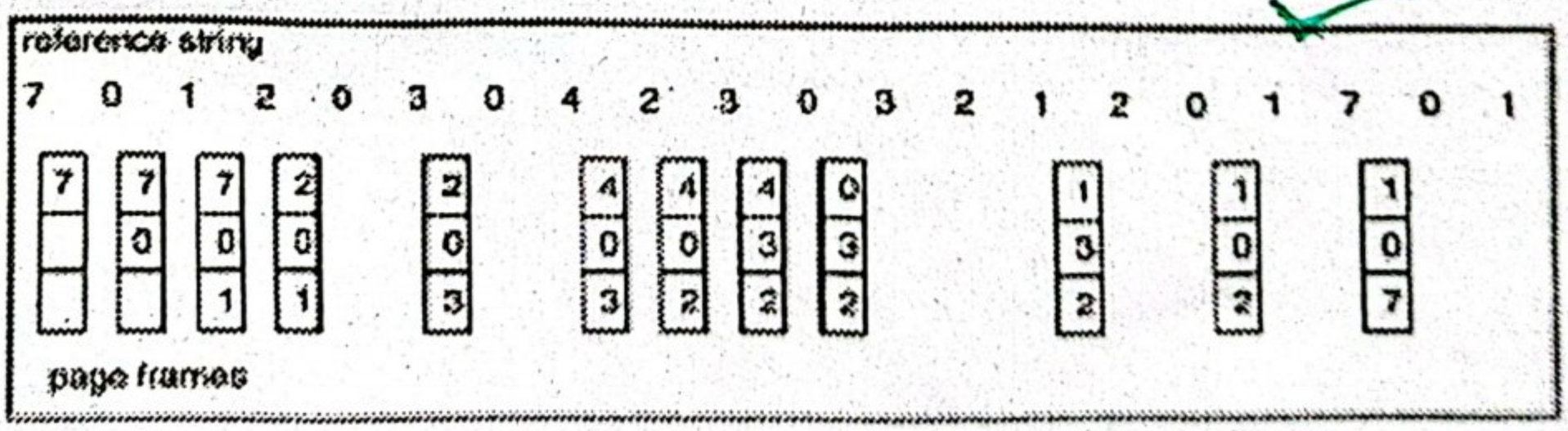


Example for the optimal page replacement algorithm

Unfortunately this algorithm is difficult to implement because it requires future knowledge of the reference string. As a result this algorithm is used mainly for comparison.

LRU Page Replacement

If we use the recent past as an approximation of the near future, then we will replace the page that has not been used for the longest period of time. This approach is the least recently used algorithms. The following example illustrates the working of LRU algorithm.



Example for the LRU page replacement algorithm

Here is another example, which uses the same reference string as used in the examples for the FIFO and optimal replacement algorithms. The number of page faults in this case is nine.

CamScanner

Operating Systems
Lecture No. 40

Reading Material

- Chapter 10 of the textbook
- Lecture 40 on Virtual TV

Summary

- Belady's Anomaly
- Page Replacement Algorithms
  - Least Frequently Used (LFU)
  - Most Frequently Used (MFU)
  - Page Buffering Algorithm
- Allocation of Frames
- Minimum Number of Frames
- Thrashing

Belady's Anomaly

Consider the following example of the FIFO algorithm.

- Number of frames allocated = 3
- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Number of page faults = 9

1	1	1	4	4	4	5	5	5
	2	2	2	1	1	1	3	3
		3	3	3	2	2	2	4

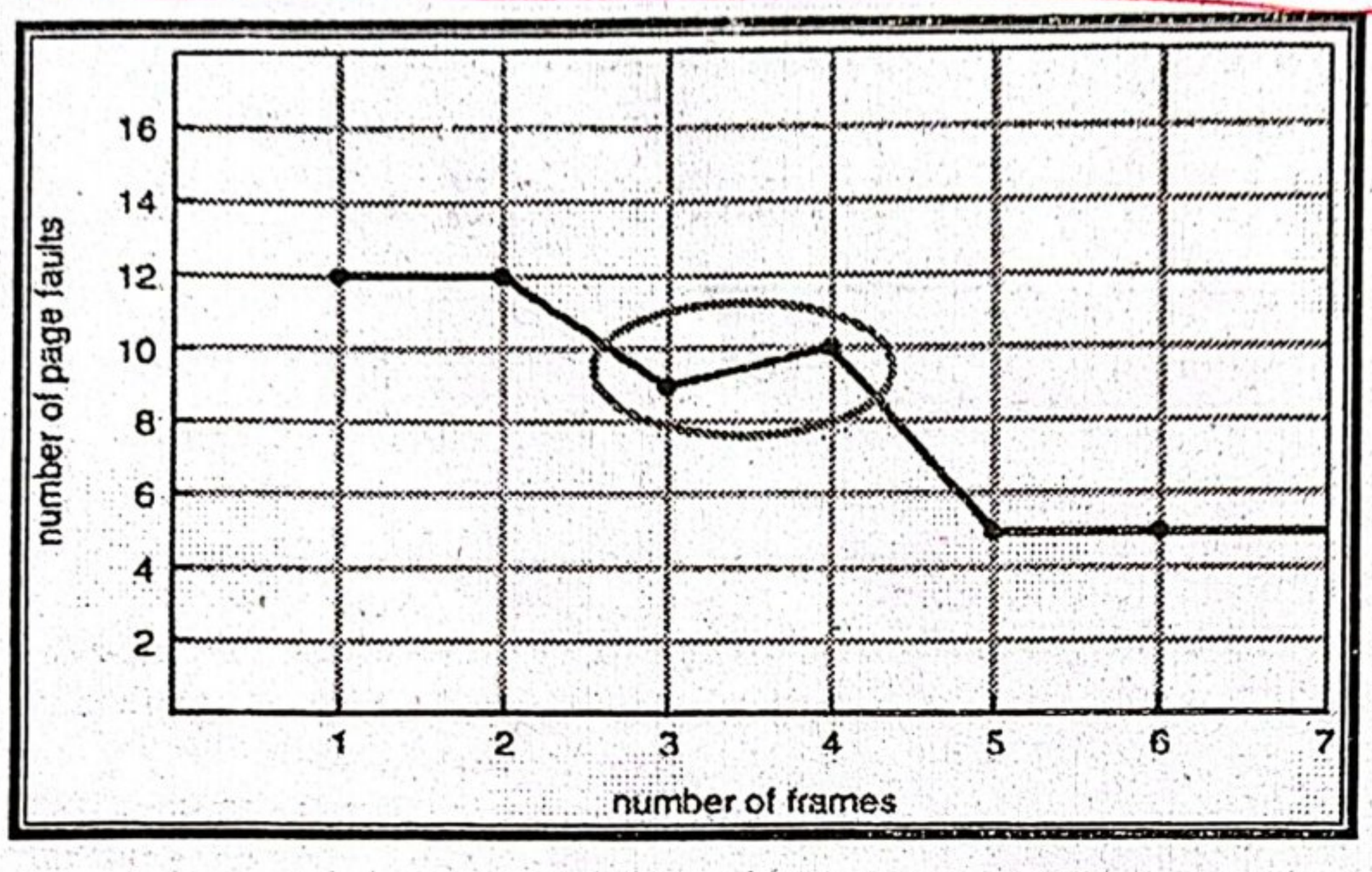
Now an intuitive idea is that if we increase the number of frames allocated to 4 from 3, the page faults should decrease, but the following example demonstrates otherwise.

- Number of frames allocated = 4
- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Number of page faults = 10

1	1	1	1	5	5	5	5	4	4
	2	2	2	2	1	1	1	1	5
		3	3	3	3	2	2	2	2
			4	4	4	4	3	3	3

CamScanner

This is due to the **Belady's Anomaly** which states that "For some page replacement algorithms, the page fault rate may increase as the number of allocated frames increases."



Belady's anomaly

Stack Replacement Algorithms

These are a class of page replacement algorithms with the following property:

Set of pages in the main memory with n frames is a subset of the set of pages in memory with n+1 frames.

These algorithms do not suffer from Belady's Anomaly. An example is the LRU algorithm.

Consider the following example which shows that LRU does not suffer from Belady's anomaly for the given reference string.

- Number of frames allocated = 3
- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Number of page faults = 10

1	1	1	4	4	4	5	3	3	3
	2	2	2	1	1	1	1	4	4
		3	3	3	2	2	2	2	5

- Number of frames allocated = 4
- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Number of page faults = 8

Radwe -

CamScanner

Another modification is to keep a pool of free frames, but to remember which page was in which frame. Since the frame contents are not modified when a frame is written to disk, the old page can be reused directly from the free-frame pool if it is needed before that frame is reused. No I/O is needed in this case. When a page fault occurs we check whether the desired page is in the free-frame pool. If it is not we must select a free frame and read into it. This method is used together with FIFO replacement in the VAX/VMS operating system.

Local vs Global Replacement

If process P generates a page fault, page can be selected in two ways:

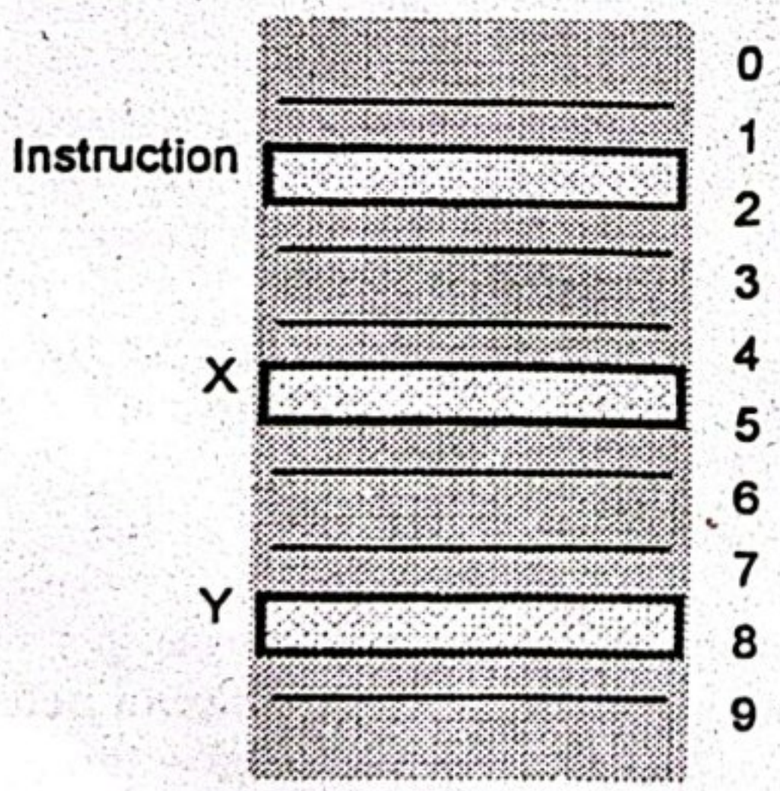
- Select for replacement one of its frames.
- Select for replacement a frame from a process with lower priority number.

Global replacement allows a process to select a replacement frame from the set of all frames, even if that frame belongs to some other process; one process can take a frame from another. Local replacement requires that each process select from only its allocated frames.

Consider an allocation scheme where we allow high priority processes to select frames from low priority processes for replacement. A process can select a replacement from among its own frames or the frames of any lower priority process. This approach allows a high priority process to increase its frame allocation at the expense of the low priority process.

Allocation of frames

Each process needs a minimum number of frames so that its execution may be guaranteed on a given machine. Let's consider the MOV X,Y instruction. The instruction is 6 bytes long (16-bit offsets) and might span 2 pages. Also, two pages to handle source and two pages are required to handle destination (assuming 16-bit source and destination).



Minimum frames required to guarantee execution of the MOV X,Y instruction

There are three major allocation schemes:

- **Fixed allocation**
  In this scheme free frames are equally divided among processes
- **Proportional Allocation**
  Number of frames allocated to a process is proportional to its size in this scheme.

CamScanner

- **Priority allocation**
  Priority-based proportional allocation

Here is an example of frame allocation:

Number of free frames = 64
Number of processes = 3
Process sizes: P1 = 10 pages; P2 = 40 pages; P3 = 127 pages

- **Fixed allocation**
  64/3 = 21 frames per process and one put in the free frames list

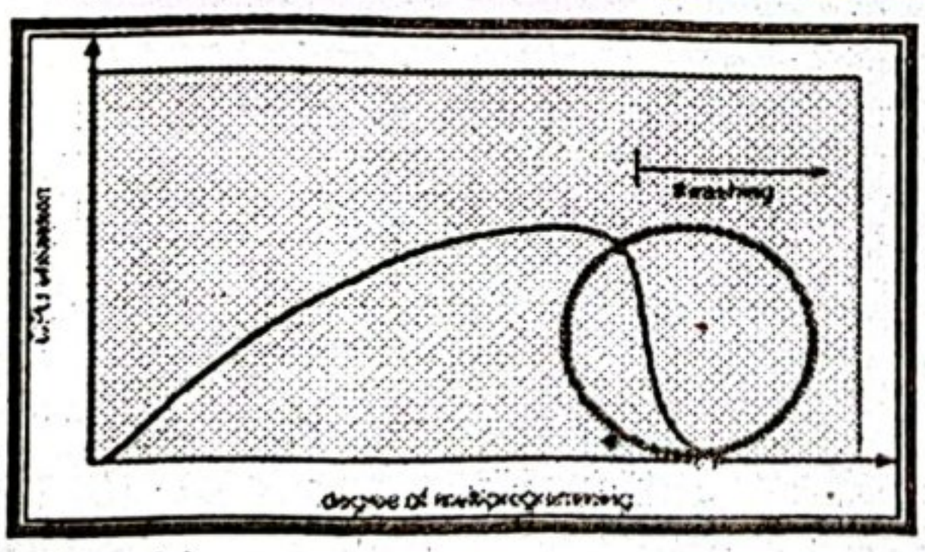
- **Proportional Allocation**
  - si = Size of process Pi
  - S = ∑ si
  - m = Number of free frames
  - ai = Allocation for Pi = (si / S) * m
    - a1 = (10 / 177) * 64 = 3 frames
    - a2 = (40 / 177) * 64 = 14 frames
    - a3 = (127 / 177) * 64 = 45 frames
  - Two free frames are put in the list of free frames

Thrashing

If a process does not have "enough" pages, the page-fault rate is very high. This leads to low CPU utilization. The operating system thinks that it needs to increase the degree of multiprogramming, because it monitors CPU utilization and find it to be decreasing due to page faults. Thus another process is added to the system and hence thrashing occurs and causes throughput to plunge.

A process is **thrashing** if it is spending more time paging (i.e., swapping pages in and out) than executing. Thrashing results in severe performance problems:

- Low CPU utilization
- High disk utilization
- Low utilization of other I/O devices



Thrashing

The figure shows that as the degree of multiprogramming increases CPU utilization also increases, although more slowly, until a maximum is reached. If the degree of multiprogramming is increased further, thrashing sets in and CPU utilization drops sharply. At this point we must decrease the degree of multiprogramming. We can limit

CamScanner

Operating Systems
Lecture No. 41

Reading Material

- Chapter 10 of the textbook
- Lecture 41 on Virtual TV

Summary

- Thrashing
- The Working Set Model
- Page Fault Frequency Model
- Other Considerations
  - Prepaging
  - Page size
  - Program structure
- Examples of Virtual Memory Systems

Thrashing

If a process does not have enough frames, it will quickly page fault. At this point, if a free frame is not available, one of its pages must be replaced so that the desired page can be loaded into the newly vacated frame. However since all its pages are in active use, the replaced page will be needed right away. Consequently it quickly faults again and again. The process continues to fault, replacing pages for which it then faults and brings back in right away. This high paging activity is called **thrashing**. In this case, *only one process is thrashing*. A process is thrashing if it is spending more time paging than executing.

Thrashing results on severe performance problems. The operating system monitors CPU utilization and, if CPU utilization is too low, the operating system increases the degree of multiprogramming by introducing one or more new processes to the system. This decreases the number of frames allocated to each process currently in the system, causing more page faults and further decreasing the CPU utilization. This causes the operating system to introduce more processes into the system. As a result CPU utilization drops even further and the CPU scheduler tries to increase the degree of multiprogramming even more. Thrashing has occurred and system throughput plunges. The page fault rate increases tremendously. As a result the effective memory access time increases. Along with low CPU utilization, there is high disk utilization. There is low utilization of other I/O devices. No work is getting done, because the processes are spending all their time paging and the system spend most of its time servicing page fault. Now *the whole system is thrashing*—the CPU utilization plunges to almost zero, the paging disk utilization becomes very high, and utilization of other I/O devices becomes very low.

If a global page replacement algorithm is used, it replaces pages with no regard to the process to which they belong. Now suppose that a process enters a phase in its execution and needs more frames. It starts faulting and taking frames away from other processes. These processes need those pages however and so they also fault taking frames away

CamScanner

Operating Systems
Lecture No. 42

Reading Material
- Chapter 11 of the textbook
- Lecture 42 on Virtual TV

Summary
- File Concept
- File Types
- File Operations
- Access Methods
- Directories
- Directory Operations
- Directory Structure

The File Concept

Computers can store information on several different storage media, such as magnetic disks, magnetic tapes and optical disks. The operating system abstracts from the physical properties of its storage devices to define a logical storage unit (the file). Files are mapped by the OS onto physical devices. These storage devices are usually non-volatile, so the contents are persistent through power failures, etc. A file is a named collection of related information that is recorded on secondary storage. Data cannot be written to secondary storage unless they are within a file. Commonly, files represent programs (source and object forms) and data. Data files may be numeric, alphabetic, alphanumeric or binary. In essence it is a contiguous logical address space.

File Structure

A file has certain defined structure characteristics according to its type. A few common types of file structures are:

None – file is a sequence of words, bytes

Simple record structure
- Lines
- Fixed length
- Variable length

Complex Structures
- Formatted document
- Relocatable load file

UNIX considers each file to be a sequence of bytes; no interpretation of these bytes is made by the OS. This scheme provides maximum flexibility but little support. Each application program must include its own code to interpret an input file into the

CamScanner

appropriate structure. However all operating systems must support at least one structure-that of an executable file-so that the system is able to load and run programs.

File Attributes

Every file has certain attributes, which vary from one OS to another, but typically consist of these:

Name: The symbolic file name is the only information kept in human-readable form

Type: This information is needed for those systems that support different types.

Location: This location is a pointer to a device and to the location of the file on that device.

Size: The current size of the file (in bytes, words or blocks) and possibly the maximum allowed size are included in this attribute.

Protection: Access control information determines who can do reading , writing, etc.

Owner

Time and date created: useful for security, protection and usage monitoring.

Time and date last updated: useful for security, protection and usage monitoring.

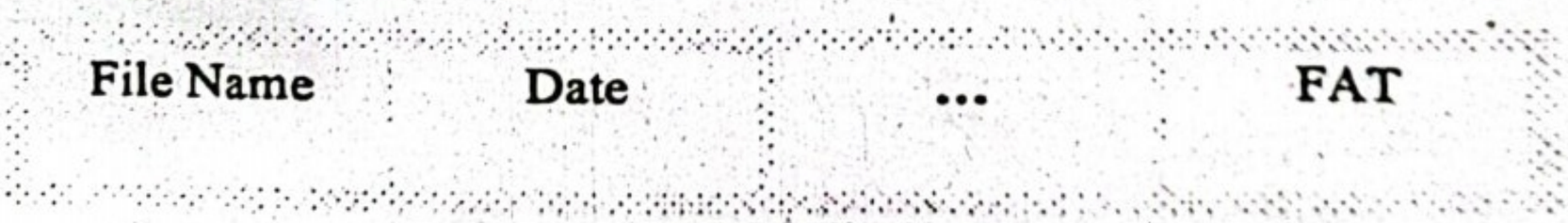
Read/write pointer value

Where are Attributes Stored?

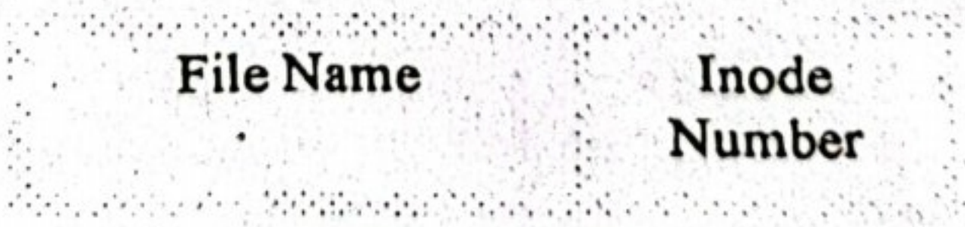
File attributes are stored in the directory structure, as part of the **directory entry** for a file, e.g., in DOS, Windows, or in a separate data structure; in UNIX/Linux this structure is known as the **inode** for the file.

Directory Entry

A file is represented in a directory by its directory entry. Contents of a directory entry vary from system to system. For example, in DOS/Windows a directory entry consists of file name and its attributes. In UNIX/Linux, a directory entry consists of file name and inode number. Name can be up to 255 characters in BSD UNIX compliant systems. Inode number is used to access file's inode. The following diagrams show directory entries for DOS/Windows and UNIX/Linux systems.



DOS/Windows



UNIX/Linux

File Operations

Various operations can be performed on files. Here are some of the commonly supported operations. In parentheses are written UNIX/Linux system calls for the corresponding operations.

CamScanner

- Create (`creat`) —two steps are necessary to create a file. First, space must be found for the file in the file system. Second, an entry for the new file must be made in the directory.
- Open (`open`) — The open operation takes a file name and searches the directory, copying the directory entry into the open-file table. The open system call can also accept access-mode information-read-only, read-write, etc. It typically returns a pointer to the entry in open-file table.
- Write (`write`) —To write to a file, we make a system call, specifying both the name of the file and the information to be written to the file. Given the name of the file, the system searches the directory to find the location of the file. The system must keep a *write* pointer to the location in the file where the next write is to take place. The write pointer must be updated whenever a write occurs.
- Read (`read`) — To read from a file we use a system call that specifies the name of the file, and where (in memory) the next block of the file should be put. The system needs top keep a read pointer to the location in the file where the next read is to take place. Once the read has taken place, the read pointer needs to be updated. A given process is usually only reading or writing to a file. The current pointer location is kept as a process **current-file-position pointer**. Both read and write use the same pointer
- Reposition within file (`lseek`) — A directory is searched for the appropriate entry and the current-file-position is set to a given value. This is often known as a file seek.
- Delete (`unlink`) — Search the directory for the named file, and then release the file space and erase the directory entry. File can be deleted using the unlink system call.
- Truncate (`creat`) — A user may want to erase the contents of the file but keep its attributes. This function allows all attributes to be unchanged except for file length., which is set to zero and file space is released. This can be achieved using creat with a special flag
- Close (`close`) — When a file is closed, the OS removes its entry in the open-file table.

File Types: Extensions

A common technique for implementing files is to include the type of the file as part of the file name. The name is split into two parts, a name and an extension, usually separated by a period character. In this way, the user and the OS can teil from the name alone, what the type of a file is.

The operating system uses the extension to indicate the type of the file and the type of operations that can be done on that file. In DOS/Windows only a file with .exe, .com, .bat extension can be executed.

The UNIX system uses a crude magic number stored at the beginning of some files to indicate roughly the type of the file-executable program, batch file/shell script, etc. Not all files have magic numbers, so system features cannot be based solely on this type of information. UNIX does allow file name extension hints, but these extensions are not enforced or depended on by the OS; they are mostly to aid users in determining the type of contents of the file. Extension can be used or ignored by a given application.

CamScanner

The following tables shows some of the commonly supported file extensions on different operating systems.

file type	usual extension	function
executable	exe, com, bin or none	read to run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rrf, doc	various word-processor formats
library	lib, a, so, dll, mpeg, mov, rm	libraries of routines for programmers
print or view	arc, zip, tar	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm	binary file containing audio or A/V information

Common file types

File Types in UNIX

UNIX does not support supports seven types of file:

- **Ordinary file**: used to store data on secondary storage device, e.g., a source program(in C), an executable program. Every file is a sequence of bytes.
- **Directory**: contains the names of other files and/or directories.
- **Block-special file**: correspond to block oriented devices such as a disk. They are used to access such hardware devices.
- **Character-special file**: correspond to character oriented devices, such as keyboard
- **Link file** (created with the `ln -s` command): is created by the system when a symbolic link is created to an existing file, allowing you to rename the existing file and share it without duplicating its contents without
- **FIFO** (created with the mkfifo or mknod commands or system calls): enable processes to communicate with each other. A FIFO(name pipe) is an area in the kernel that allows two processes to communicate with each other provided they are running on the same system , but the processes do not have to be related to each other.
- **Socket** (in BSD-compliant systems—socket): can be used by the process on the same computer or on different computers to communicate with each other.

File Access

Files store information that can be accessed in several ways:

CamScanner

Directory Operations

The following directory operations are commonly supported in contemporary operating systems. Next to each operation are UNIX system calls or commands for the corresponding operation.

- Create — `mkdir`
- Open — `opendir`
- Read — `readdir`
- Rewind — `rewinddir`
- Close — `closedir`
- Delete — `rmdir`
- Change Directory — `cd`
- List — `ls`
- Search

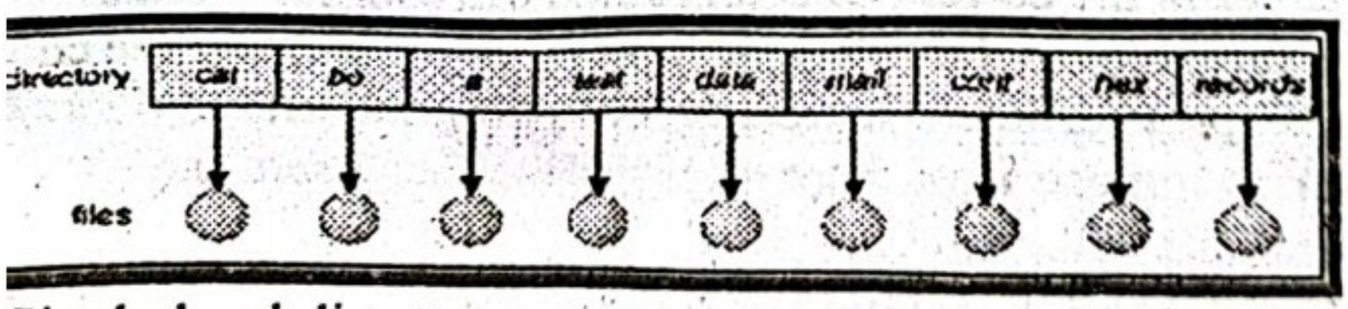
Directory Structure

When considering a particular directory structure we need to consider the following issues:

1. **Efficient Searching**
2. **Naming** – should be convenient to users
   - Two users can have same name for different files
   - The same file can have several different names
3. **Grouping** – logical grouping of files by properties, (e.g., all Java programs, all games, ..)

Single-Level Directory

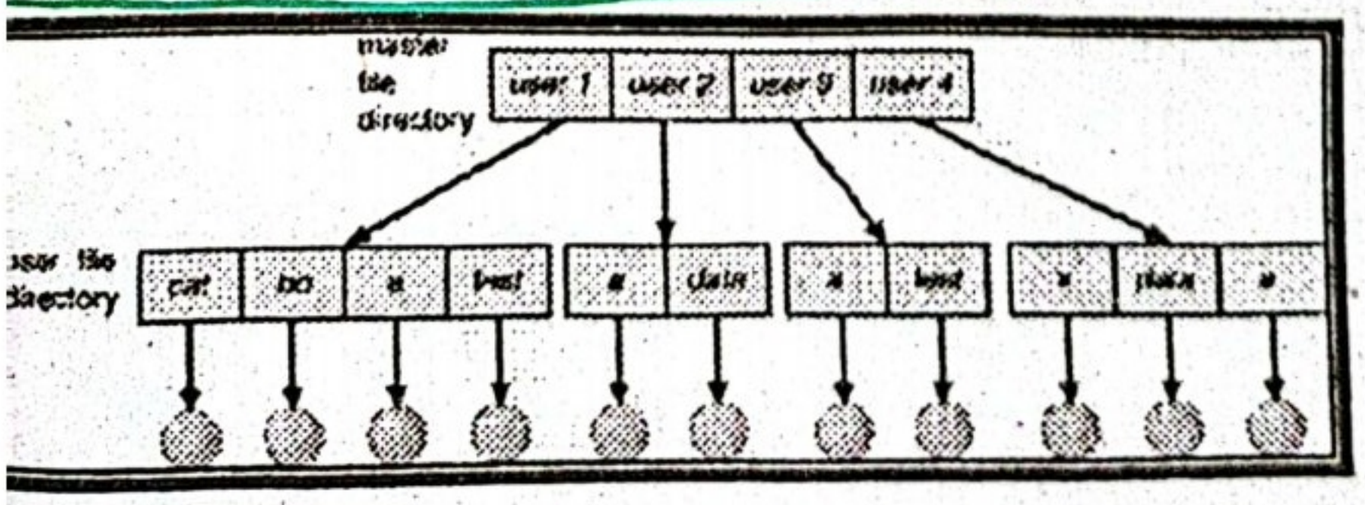
All files are contained in the same directory, which is easy to support and understand. However when the number of files increases or the system has more than one user, it has limitations. Since all the files are in the same directory, they must have unique names.



Single-level directory structure

Two-Level Directory

There is a separate directory for each user.



Two-level directory strucutre

CamScanner

are flexible. You may have soft links to directories and across file systems. However, UNIX has to support an additional file type, the link type, and a new file is created for every link, slowing down file operations.

File System Mounting

A file system is best visualized as a tree, rooted at /. /dev, /etc, /usr, and other directories in the root directory are branches, which may have their own branches, such as /etc/passwd, /usr/local, and /usr/bin. Filling up the root file system is not a good idea, so splitting /var from / is a good idea. Another common reason to contain certain directory trees on other file systems is if they are to be housed on separate physical disks, or are separate virtual disks, or CDROM drives.

Mounting makes file systems, files, directories, devices, and special files available for use at a particular location. **Mount point** is the actual location from which the file system is mounted and accessed. You can mount a file or directory if you have access to the file or directory being mounted and write permission for the mount point

There are types of mounts:

- Remote mount
- Local mount

Remote mounts are done on a remote system on which data is transmitted over a telecommunication line. **Local mounts** are mounts done on your local system.

CamScanner

The tree has a root directory. Every file in the system has a unique pathname. A path name is the path from the root, through al the subdirectories to a specified file. A directory/subdirectory contains a set of files or subdirectories. In normal use, each user has a current directory. The current directory should contain most of the files that are of current interest to the user. When a reference to a file is made, the current directory is searched. If a file is needed that is not in the current directory, then the user must either specify a path name or change the directory to the directory holding the file(using the cd system call).This structure hence supports efficient searching. Allowing the user to define his own subdirectories permits him to impose a structure on his files.A lso users can access files of other users.

UNIX / Linux Notations and Concepts Imp

- Root directory (/)
- Home directory
  - ~, $HOME, $home
  - cd ~
  - cd
- Current/working directory (.)
  - pwd
- Parent of Current Directory (..)
- Absolute Pathname
  - Starts with the root directory
  - For example, /etc, /bin, /usr/bin, /etc/passwd, /home/students/ibraheem
- Relative Pathname

CamScanner

reference count means no more references to the file/directory can be deleted. However, cycles can exist, e.g, due to self-referencing. In this case we need to use a garbage collection scheme, which involves traversing the entire file system, marking everything that can be accessed. Then a second pass collects everything that is not marked onto a lis of free space. However this is extremely time consuming and is seldom used. However i is necessary because of possible cycles in a graph.

Links in UNIX

Most Imp

UNIX supports two types of links:

- **Hard links**
- **Soft (symbolic) links**

The ln command is used to create both links, ln –s is used to create a soft link

- `ln [options] existing-file new-file`
- ln [options] existing-file-list directory

CamScanner

```
/oldexport     /dev/dsk/c0t8d0   4668856    4229110    393058     92%
/export/ho     /dev/dsk/c0t12d  23684712   21714309   1733556     93%
$
```

File Sharing

Sharing of files on multi-user systems is desirable. People working on the same project need to share information. For instance: software engineers working on the same project need to share files or directories related to the project

Sharing may be done through

1. **Duplicating files:** Make copies of the file and give them to all team members. This scheme works well if members of the team are to work on these shared files sequentially. If they work on the files simultaneously, the copies become inconsistent and no single copy reflects the works done by all members. However it is simple to implement.
2. **Common login** for members of a team: The system admin creates a new user group and gives the member access to the new account. All files and directories created by any team member under this account and are owned by the team. This works well if number of teams is small and teams are stable. However a separate account is needed for the current project and the system administrator has to create a new account for every team
3. Setting appropriate **access permissions.** Team members put all shared files under one member's account and the access permissions are set so all the members can access it. This scheme works well if *only* this team's members form the user group. File access permissions can be changed using the chmod system call:

```
chmod [options] octal-mode file list
chmod [options] symbolic -mode file-list
```


CamScanner

perating Systems
ecture No. 44

eading Material
- Chapters 11 and 12 of the textbook
- Lecture 44 on Virtual TV

ummary
- File Protection
- In-Memory Data Structures
- Space Allocation Techniques
- Contiguous, Linked, Index

rotection
he need to protect files is a direct result of the ability to access files. Systems that do not ermit access to the files of other users do not need protection. Thus we could provide omplete protection by prohibiting access. Alternatively we could provide free access ith no protection. Both approaches are too extreme for general use. What is needed is ontrolled access. File owner/creator should be able to control
- What can be done
- By whom

everal types of operations may be controlled:
- Read: read from the file
- Write: write or rewrite to the file
- Execute: Load the file into memory and execute it
- Append: Write new information at the end of the file
- Delete: Delete the file and free its space for possible reuse
- List: List the name and attributes of the file

UNIX Protection
UNIX recognizes three modes of access: **read**, **write**, and **execute** (r, w, x). The execute permission on a directory specifies permission to **search** the directory.

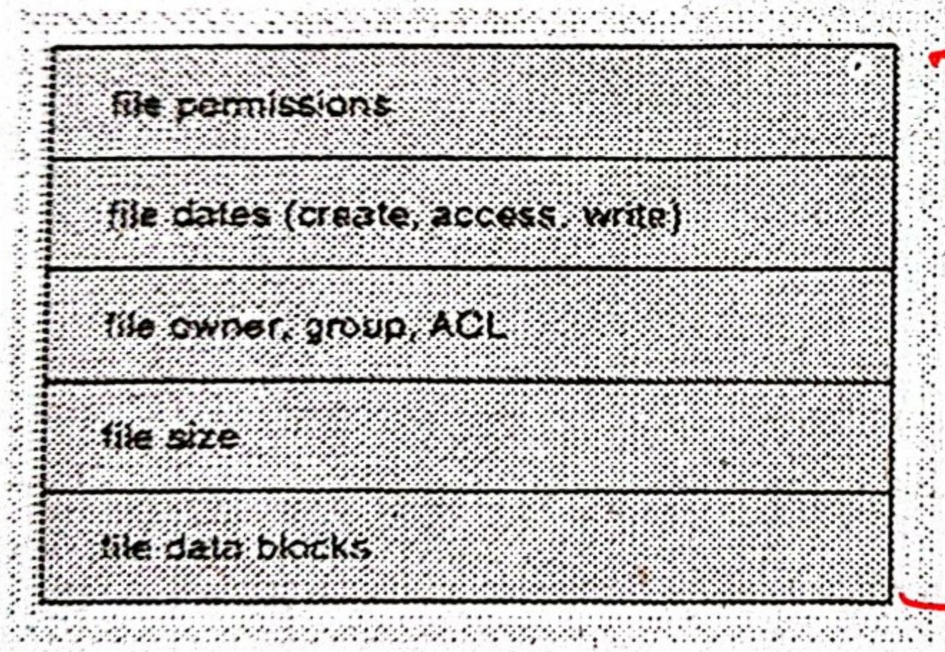
The three classes of users are:
- **Owner:** user is the owner of the file
- **Group:** someone who belongs to the same group as the owner
- **Others:** everyone else who has an account on the system

A user's access to a file can be specified by an octal digit. The first bit of the octal digit specifies the read permission, the second bit specifies the write permission, and the third bit specifies the execute permission. A bit value 1 indicates permission for access and 0 indicates no permission. Here is an example:

CamScanner

File Control Block

A file control block is a memory data structure that contains most of the attributes of a file. In UNIX, this data structure is called inode (for index node). Here are possible values in this data structure.



UNIX inode

In-Memory Data Structures

CamScanner

The open call passes a file name to the file system. When a file is opened, the directory structure is searched for the given file name and file's inode. An entry is made in the per process open-file table (aka the file descriptor table), with a pointer to the entry in the system wide open file table. The system wide open file table contains the pointer to the current location in the file and a pointer to file's inode. The open call returns an index for the appropriate entry in the per-process file system table. All file operations are performed via this index, which is called the file descriptor in UNIX/Linux jargon.

Space Allocation Methods

We now turn to some file system implementation issues, in particular space allocation techniques and free space management methods. Here are the three commonly used methods for file space allocation.

- Contiguous allocation
- Linked allocation
- Indexed allocation

Contiguous Allocation

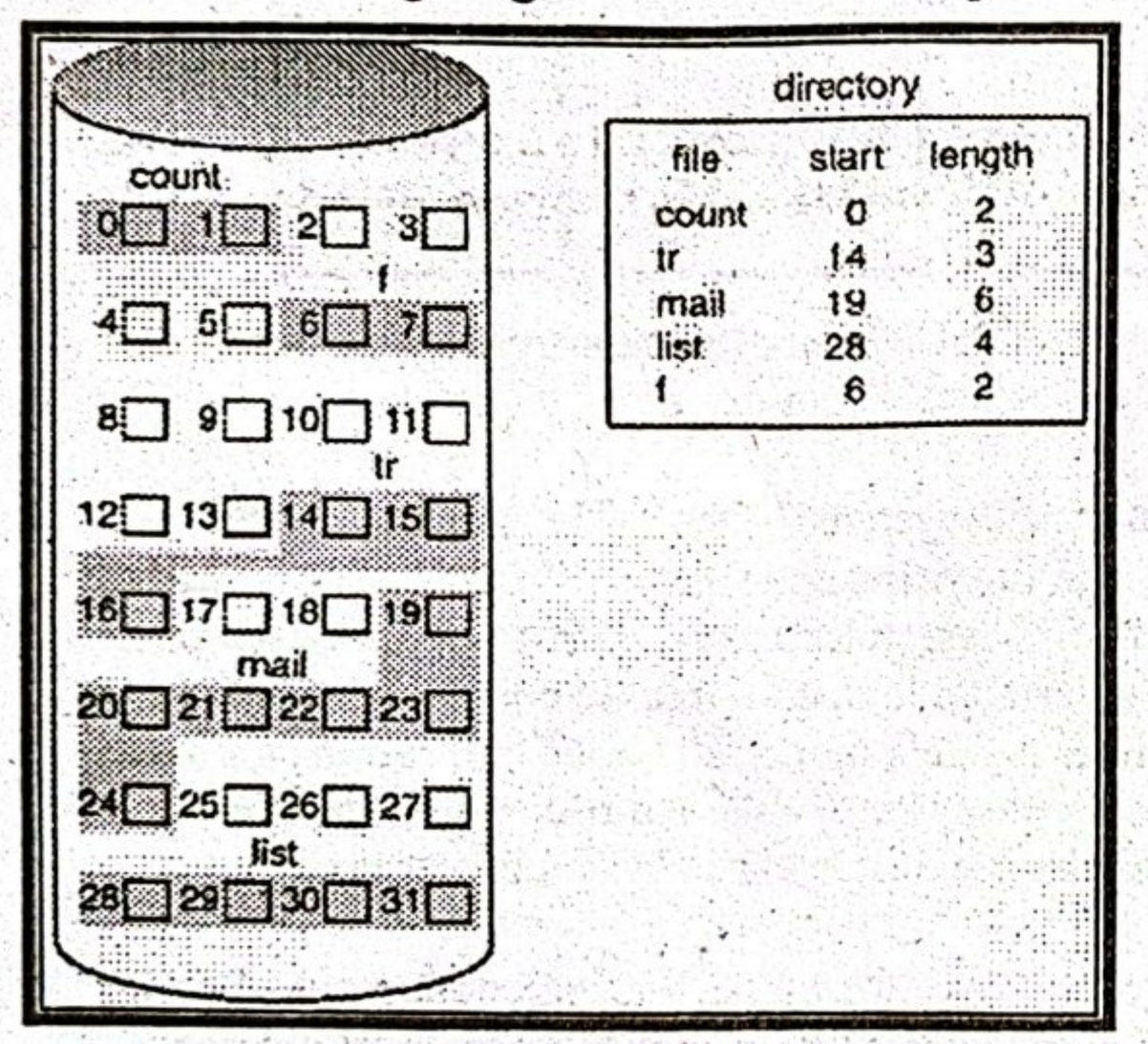
The contiguous allocation method requires each file to occupy a set of contiguous blocks on the disk. The directory entry for each file contains starting block number and file size (in blocks). Disk addresses define a linear ordering on the disk. With this ordering, assuming only one job is accessing the disk, accessing b+1 block after block b normally requires no head movement. When head movement is needed it is only one track. Both sequential and direct access can be supported by contiguous allocation. For direct access ... that starts at block b we can immediately access block b+i.

Best-fit, first-fit, or worst-fit algorithms are the strategies used to select a hole from the set of available holes. Neither first fit, nor best fit is clearly best in terms of both time and storage utilization, but first fit is generally faster.

These algorithms suffer from the problem of external fragmentation. As files are allocated or deleted, the free disk is broken into little pieces. This situation results in **external fragmentation** of disk (similar to external fragmentation of main memory due to segmentation). Disk defragmenter utility needs to be used for removing external fragmentation.

Determining how much space is needed for a file is another problem. User needs to declare file size, and estimating file size may be difficult. Also file growth is expensive in contiguous allocation. Worst-fit space allocation algorithm can be used to allow growth in a file's size.

The following diagram shows an example of the contiguous allocation scheme.



Contiguous allocation

Linked Allocation

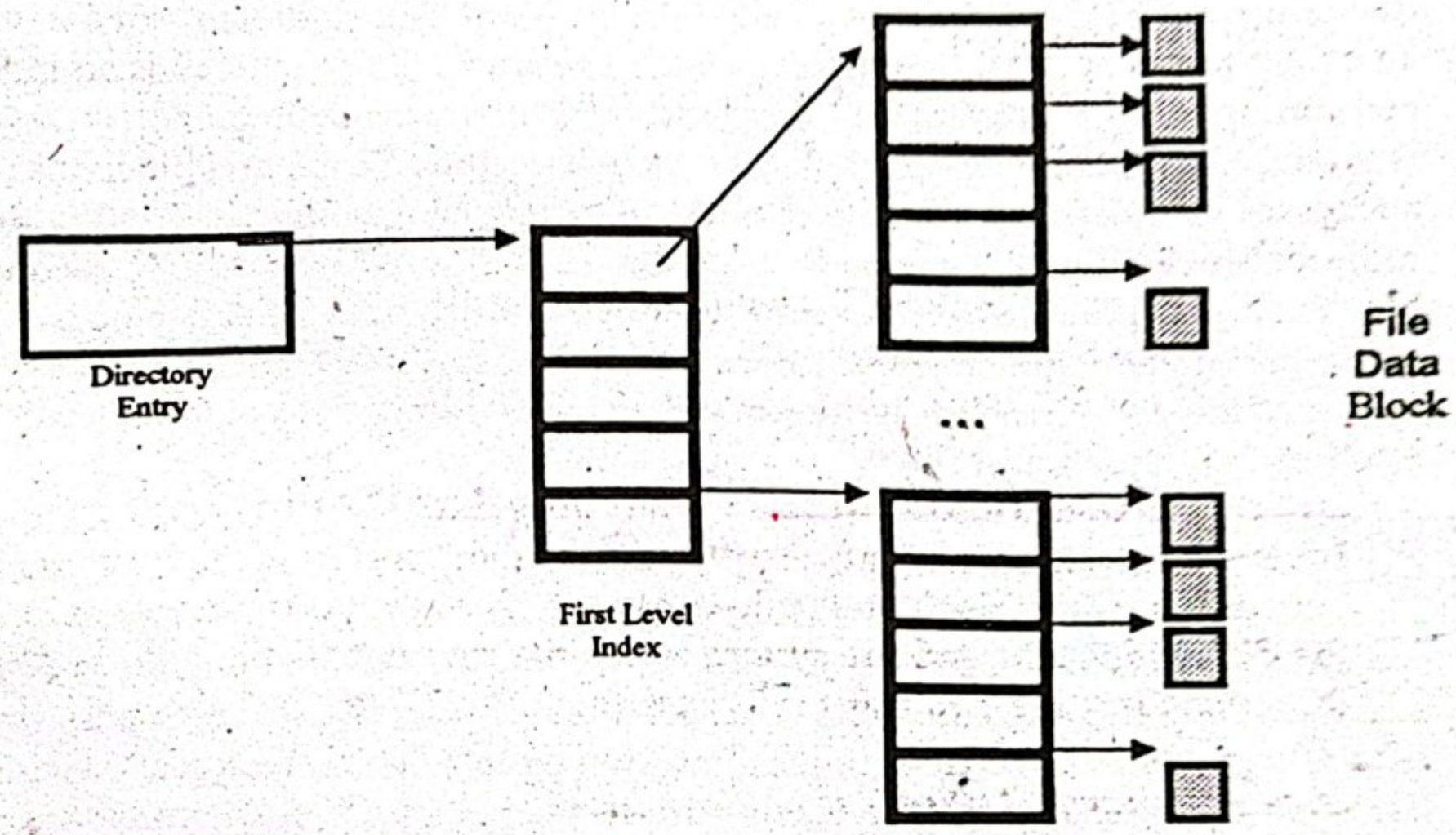
Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file. There is no external fragmentation with linked allocation, and any free block on the free-space list can be used to satisfy a request. There is no wastage of space. However, a major disadvantage with linked allocation is that it can be used only for sequential access files. To find the *i*th block of a file, we must start at the beginning of that file and follow the pointers until we get back to the *i*th block. Consequently it is inefficient to support a direct access capability for linked allocation files.

Here is an example of linked allocation.

CamScanner

Multi-level index scheme

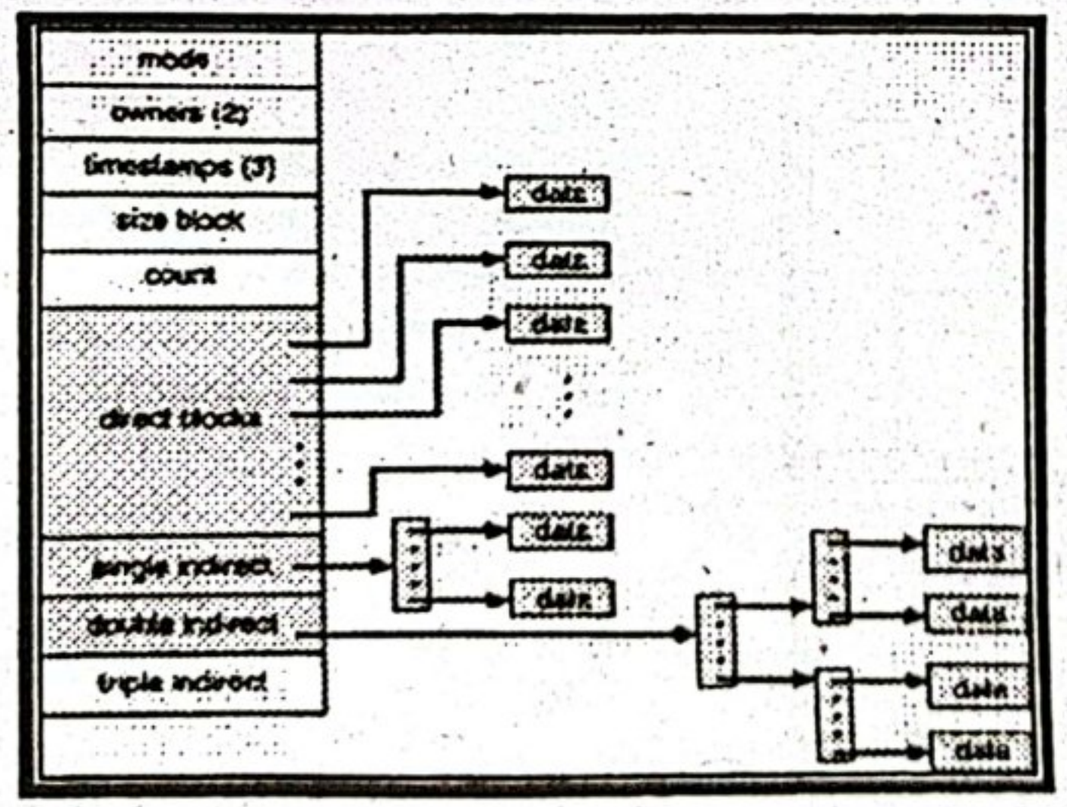
The second method of handling multiple index blocks is to maintain multi-level indexing. In the following diagram, we show two-level index table.



Two level Index Table

UNIX Space Allocation

The UNIX file manager uses a combination of indexed allocation and linked lists for the index table. It maintains 10-15 direct pointers to file blocks, and three indirect pointers (one-level indirect, two-level indirect, and three-level indirect), all maintained in file's inode, as shown below.



UNIX inode

Let's consider a UNIX system with following attributes:

CamScanner

depends on the size of the blocks and the size of the disk addresses used in the system. The next pointers point to indirect blocks. The single indirect block is an index block containing not the data but rather the addresses of blocks that do contain data. Then there is a double indirect block pointer, which contains the address of a block that contains the addresses of blocks that contain pointers to the actual data blocks. Finally, the triple indirect block pointer points to first-level index block, which points to second-level index blocks, which point to third-level index blocks, which point to data blocks. With the given parameters, the maximum file size will be $[10 + 1024 + 1024^2 + 1024^3]$ blocks—multiply this by the block size to get size in bytes. Similarly, you can calculate the pointer overhead for the largest file.

File Allocation Table (FAT)

The file system on an MS-DOS floppy disk is based on **file allocation table** (FAT) file system in which the disk is divided into a reserved area (containing the boot program) and the actual file allocation tables, a root directory and file space. Space allocated for files is represented by values in the allocation table, which effectively provide a linked list of all the blocks in the file. Each entry is indexed by a block number and value in a table location contains block number for the next file block. First block number for a file is contained in file's directory entry. Special values designate end of file, unallocated and bad blocks. The following diagram summarizes the overall picture of FAT.